

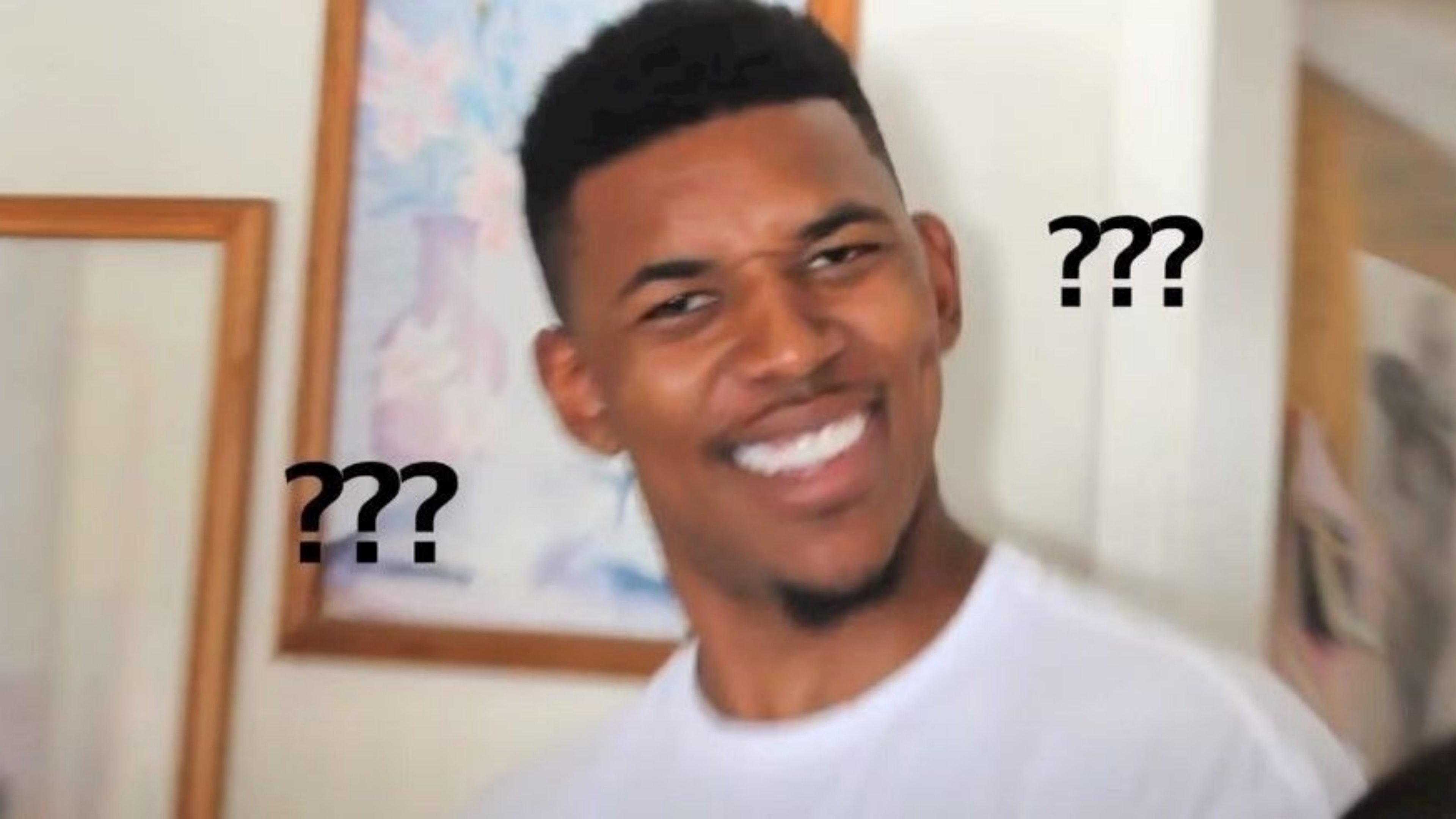
В погоне за наносекундами:

Узкие места Golang глазами C++-
разработчика

Владимир Пустовалов

C++ backend developer





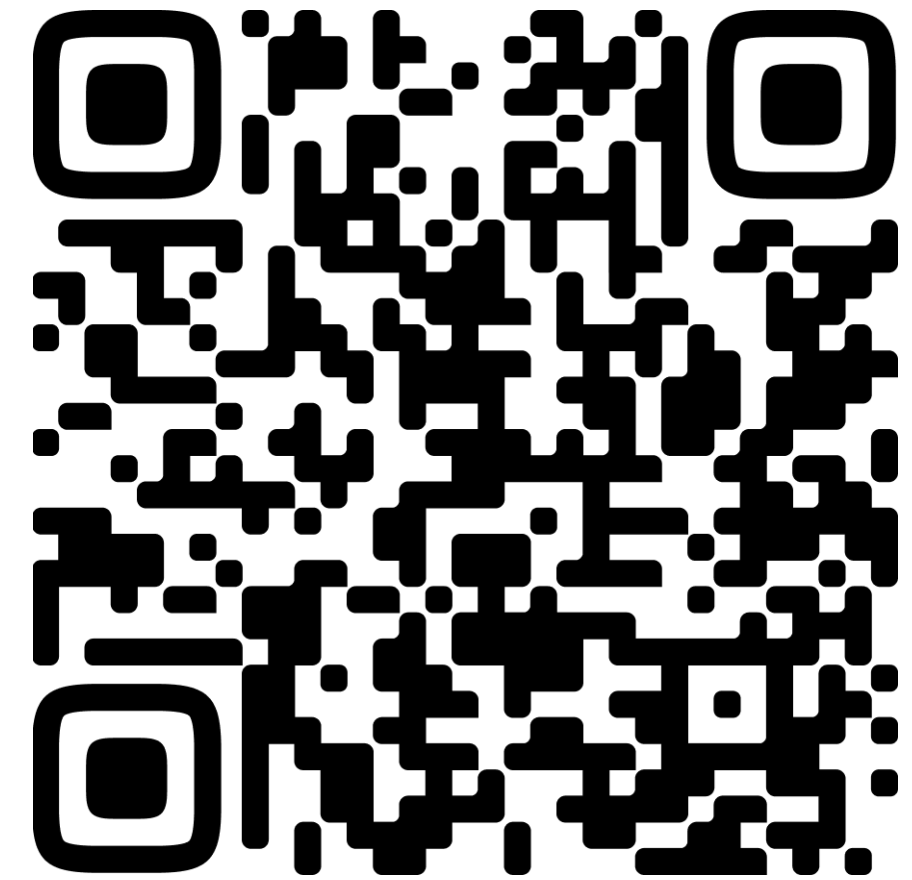
???

???

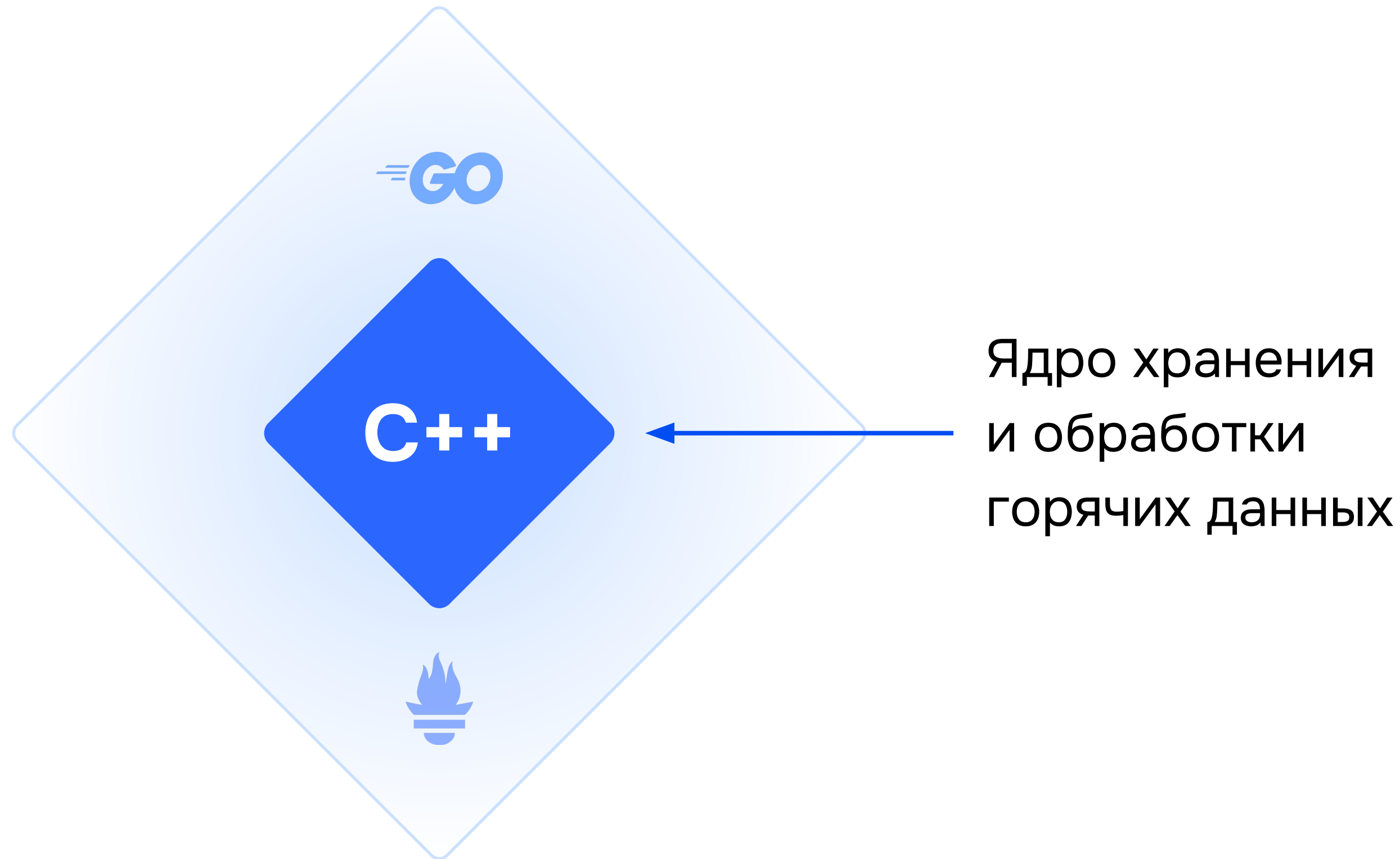


Deckhouse Prom++

Prom++ — **ОПТИМИЗИРОВАННЫЙ**
Prometheus с потреблением
памяти до 10 раз меньше



Архитектура Prom++



Через ассемблер к плюсам: устройство и оптимизация CGo



Habr



YouTube

```
func iterateInterface(factory BlockIteratorFactory) (sum int64, err error) {
    iterator := factory.Iterator()
    defer iterator.Close()

    for b := range iterator.All() {
        if blockDuration, err := getBlockDuration(b); err != nil {
            return sum, err
        } else {
            sum += blockDuration
        }
    }

    return
}

func getBlockDuration(block BlockInterface) (int64, error) {
    ...
}
```

Code 1

```
func iterateInterface(factory BlockIteratorFactory) (sum int64, err error) {
    iterator := factory.Iterator()
    defer iterator.Close()

    for b := range iterator.All() {
        if blockDuration, err := getBlockDuration(b); err != nil {
            return sum, err
        } else {
            sum += blockDuration
        }
    }

    return
}

func getBlockDuration(block BlockInterface) (int64, error) {
    ...
}
```

Code 1

```
func iterateInterface(factory BlockIteratorFactory) (sum int64, err error) {
    iterator := factory.Iterator()
    defer iterator.Close()

    for b := range iterator.All() {
        if blockDuration, err := getBlockDuration(b); err != nil {
            return sum, err
        } else {
            sum += blockDuration
        }
    }

    return
}

func getBlockDuration(block BlockInterface) (int64, error) {
    ...
}
```

Code 1

```
func iterateInterface(factory BlockIteratorFactory) (sum int64, err error) {
    iterator := factory.Iterator()
    defer iterator.Close()

    for b := range iterator.All() {
        if blockDuration, err := getBlockDuration(b); err != nil {
            return sum, err
        } else {
            sum += blockDuration
        }
    }

    return
}

func getBlockDuration(block BlockInterface) (int64, error) {
    ...
}
```

Code 1

```
func iterateInterface(factory BlockIteratorFactory) (sum int64, err error) {
    iterator := factory.Iterator()
    defer iterator.Close()

    for b := range iterator.All() {
        if blockDuration, err := getBlockDuration(b); err != nil {
            return sum, err
        } else {
            sum += blockDuration
        }
    }

    return
}

func getBlockDuration(block BlockInterface) (int64, error) {
    ...
}
```

Code 1

```

func iterateRaw(factory BlockIteratorFactory) (sum int64, err error) {
    iterator := factory.Iterator()

    var mint, maxt int64
    for iterator.Next() {
        if b, ok := iterator.At().(block); ok {
            if m, ok := b.Meta().(meta); ok {
                mint, maxt, err = m.Bounds()
            } else {
                mint, maxt, err = b.Meta().Bounds()
            }
        } else {
            mint, maxt, err = iterator.At().Meta().Bounds()
        }

        if err != nil {
            iterator.Close()
            return sum, err
        }

        sum += maxt - mint
    }

    iterator.Close()
    return
}

```

Code 2

```

func iterateRaw(factory BlockIteratorFactory) (sum int64, err error) {
    iterator := factory.Iterator()

    var mint, maxt int64
    for iterator.Next() {
        if b, ok := iterator.At().(block); ok {
            if m, ok := b.Meta().(meta); ok {
                mint, maxt, err = m.Bounds()
            } else {
                mint, maxt, err = b.Meta().Bounds()
            }
        } else {
            mint, maxt, err = iterator.At().Meta().Bounds()
        }

        if err != nil {
            iterator.Close()
            return sum, err
        }

        sum += maxt - mint
    }

    iterator.Close()
    return
}

```

Code 2

```

func iterateRaw(factory BlockIteratorFactory) (sum int64, err error) {
    iterator := factory.Iterator()

    var mint, maxt int64
    for iterator.Next() {
        if b, ok := iterator.At().(block); ok {
            if m, ok := b.Meta().(meta); ok {
                mint, maxt, err = m.Bounds()
            } else {
                mint, maxt, err = b.Meta().Bounds()
            }
        } else {
            mint, maxt, err = iterator.At().Meta().Bounds()
        }

        if err != nil {
            iterator.Close()
            return sum, err
        }

        sum += maxt - mint
    }

    iterator.Close()
    return
}

```

Code 2

```

func iterateRaw(factory BlockIteratorFactory) (sum int64, err error) {
    iterator := factory.Iterator()

    var mint, maxt int64
    for iterator.Next() {
        if b, ok := iterator.At().(block); ok {
            if m, ok := b.Meta().(meta); ok {
                mint, maxt, err = m.Bounds()
            } else {
                mint, maxt, err = b.Meta().Bounds()
            }
        } else {
            mint, maxt, err = iterator.At().Meta().Bounds()
        }

        if err != nil {
            iterator.Close()
            return sum, err
        }

        sum += maxt - mint
    }

    iterator.Close()
    return
}

```

Code 2

```

func iterateRaw(factory BlockIteratorFactory) (sum int64, err error) {
    iterator := factory.Iterator()

    var mint, maxt int64
    for iterator.Next() {
        if b, ok := iterator.At().(block); ok {
            if m, ok := b.Meta().(meta); ok {
                mint, maxt, err = m.Bounds()
            } else {
                mint, maxt, err = b.Meta().Bounds()
            }
        } else {
            mint, maxt, err = iterator.At().Meta().Bounds()
        }

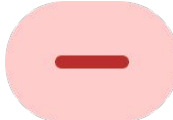
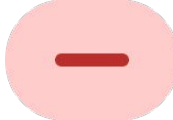
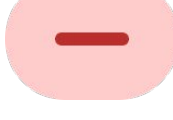
        if err != nil {
            iterator.Close()
            return sum, err
        }

        sum += maxt - mint
    }

    iterator.Close()
    return
}

```

Code 2

	Code 1	Code 2
Defer		
Range over function		
Разбиение кода на функции		
Интерфейсы		



Бенчмарк

Время

Code1

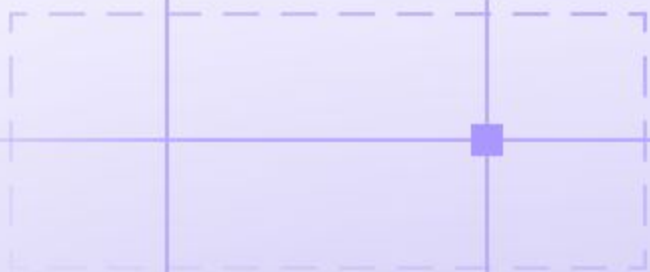
33.25 ms

Code2

3.28 ms



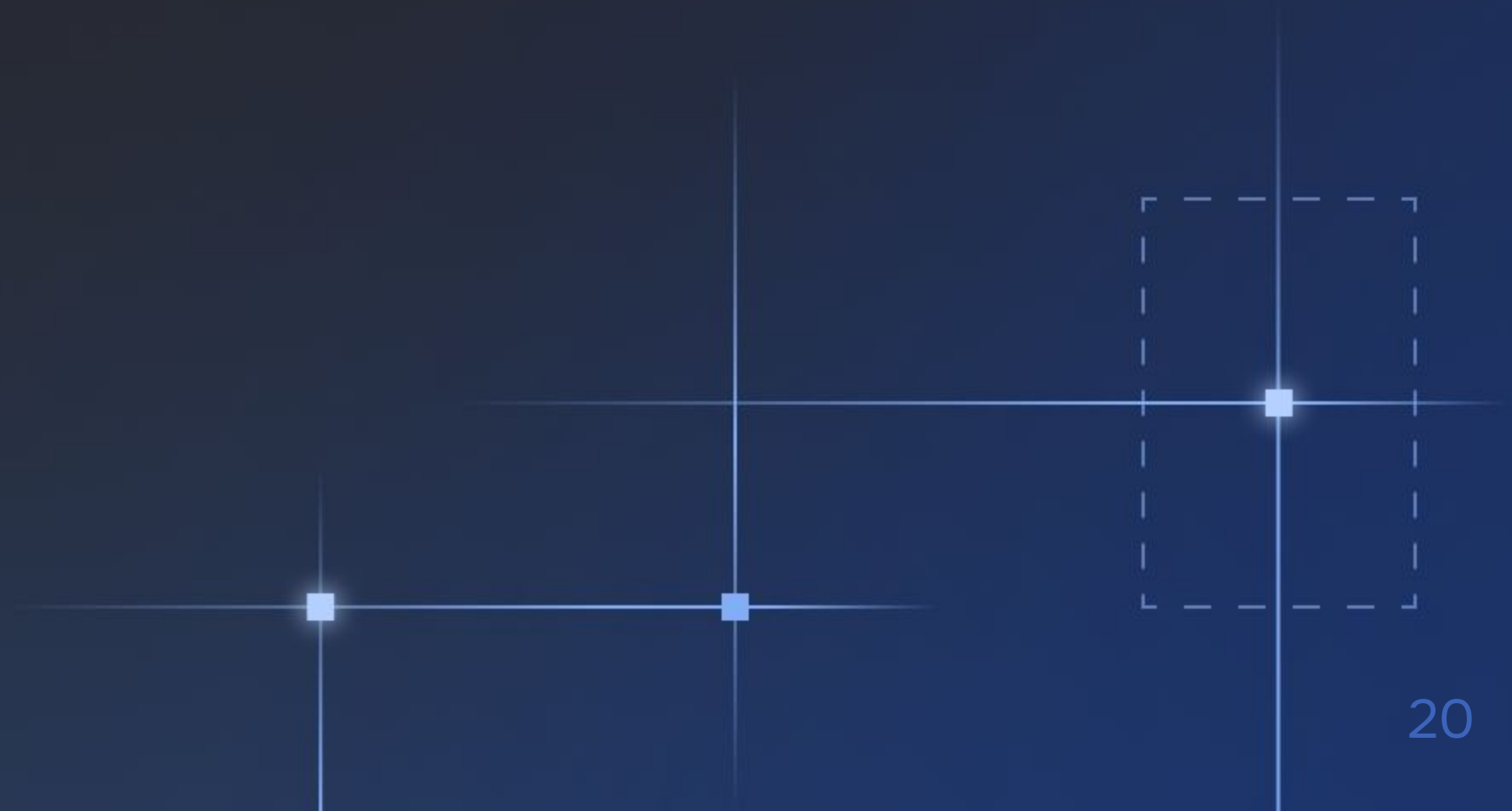
Просадка производительности ~ в 11 раз



О чём пойдёт речь в докладе:

- 01 Функции
- 02 Интерфейсы
- 03 Range over function
- 04 Defer/Lambda

Дисклеймер



Benchmarking



Бенчмаркинг —

это процесс измерения и сравнения
производительности различных алгоритмов

Бенчмаркинг

- ✓ ARM Neoverse N-1 @ 3.0GHz, Ubuntu 22.04.5
- ✓ Go 1.26.5 linux/arm64
- ✓ Testing бенчмарк

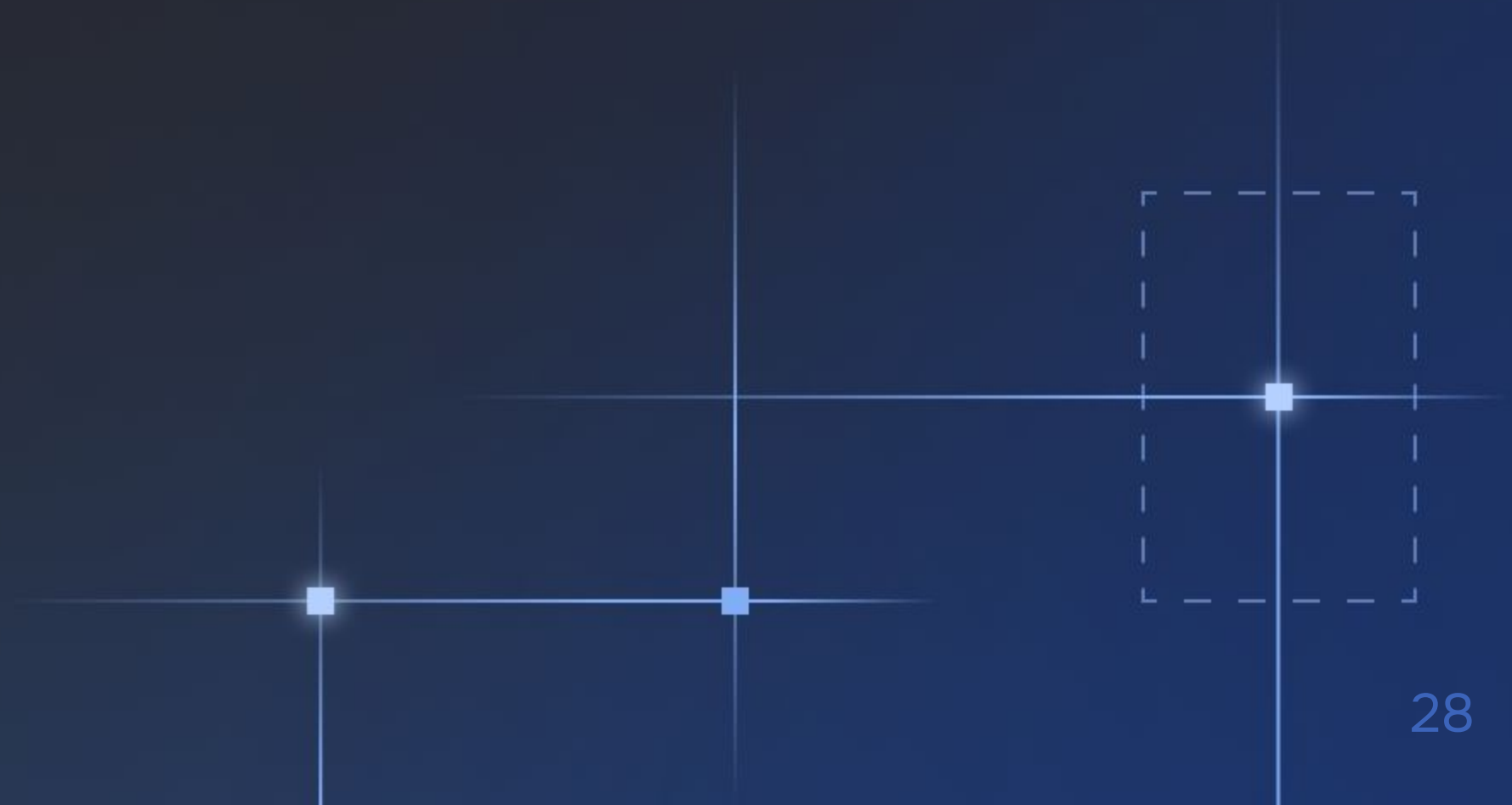
```
GOGC=off && \  
nice -n -20 \  
go test . -bench=. -count=10 | tee bench.txt && \  
benchstat bench.txt
```

```
GOGC=off && \  
nice -n -20 \  
go test . -bench=. -count=10 | tee bench.txt && \  
benchstat bench.txt
```

```
GOGC=off && \  
nice -n -20 \  
go test . -bench=. -count=10 | tee bench.txt && \  
benchstat bench.txt
```

```
GOGC=off && \  
nice -n -20 \  
go test . -bench=. -count=10 | tee bench.txt && \  
benchstat bench.txt
```

Функции



Go inlining

```
//go:noinline  
func GoFunction() {}
```

Структура Go-функции

```
func sum(i int, n []int) int {  
  
    sum := 0  
    for ; i < len(n); i++ {  
        sum += n[i]  
    }  
    return sum  
  
}
```

Структура Go-функции

```
func sum(i int, n []int) int {  
    sum := 0  
    for ; i < len(n); i++ {  
        sum += n[i]  
    }  
    return sum  
}
```

```
TEXT sum(SB)  
    MOVD 16(R28), R16  
    CMP R16, RSP  
    BLS 88  
    ...  
  
    MOVD (R1)(R0<<3), R4  
    ADD R3, R4, R3  
    ADD $1, R0, R0  
    CMP R0, R2  
    ...  
  
    STP (R0, R1), 8(RSP)  
    STP (R2, R3), 24(RSP)  
    MOVD R30, R3  
    CALL runtime.morestack_noctxt
```

Структура Go-функции

```
func sum(i int, n []int) int {  
    // prologue  
    sum := 0  
    for ; i < len(n); i++ {  
        sum += n[i]  
    }  
    return sum  
}
```

01 Пролог

```
TEXT sum(SB)  
    MOVD 16(R28), R16  
    CMP R16, RSP  
    BLS 88  
    ...  
  
    MOVD (R1)(R0<<3), R4  
    ADD R3, R4, R3  
    ADD $1, R0, R0  
    CMP R0, R2  
    ...  
  
    STP (R0, R1), 8(RSP)  
    STP (R2, R3), 24(RSP)  
    MOVD R30, R3  
    CALL runtime.morestack_noctxt
```

Структура Go-функции

```
func sum(i int, n []int) int {  
    // prologue  
    sum := 0  
    for ; i < len(n); i++ {  
        sum += n[i]  
    }  
    return sum  
}
```

01 Пролог

02 Тело функции

```
TEXT sum(SB)  
    MOVD 16(R28), R16  
    CMP R16, RSP  
    BLS 88  
    ...  
  
    MOVD (R1)(R0<<3), R4  
    ADD R3, R4, R3  
    ADD $1, R0, R0  
    CMP R0, R2  
    ...  
  
    STP (R0, R1), 8(RSP)  
    STP (R2, R3), 24(RSP)  
    MOVD R30, R3  
    CALL runtime.morestack_noctxt
```

Структура Go-функции

```
func sum(i int, n []int) int {  
    // prologue  
    sum := 0  
    for ; i < len(n); i++ {  
        sum += n[i]  
    }  
    return sum  
    // epilogue  
}
```

```
TEXT sum(SB)  
    MOVD 16(R28), R16  
    CMP R16, RSP  
    BLS 88  
    ...  
  
    MOVD (R1)(R0<<3), R4  
    ADD R3, R4, R3  
    ADD $1, R0, R0  
    CMP R0, R2  
    ...  
  
    STP (R0, R1), 8(RSP)  
    STP (R2, R3), 24(RSP)  
    MOVD R30, R3  
    CALL runtime.morestack_noctxt
```

01 Пролог

02 Тело функции

03 Эпилог

Пролог/эпилог Go-функции

01 Рост стека

Пролог/эпилог Go-функции

- 01 Рост стека
- 02 Переключение горютины

Пролог/эпилог Go-функции

```
//go:noinline  
func SumFunc(a, b int) int {  
    return a + b  
}
```

Пролог/эпилог Go-функции

```
//go:noinline
func SumFunc(a, b int) int {
    return a + b
}

func TestSumFunc(t *testing.T) {
    assert.Equal(t, 1, SumFunc(0, 1))
}
```

Пролог/эпилог Go-функции

```
//go:noinline
func SumFunc(a, b int) int {
    return a + b
}

func TestSumFunc(t *testing.T) {
    assert.Equal(t, 1, SumFunc(0, 1))
}

go test -gcflags=-S
```

Пролог/эпилог Go-функции



```
TEXT SumFunc(SB)
    ADD R1, R0, R0
    RET
```

```
//go:noinline
func SumFunc(a, b int) int {
    return a + b
}
```

Пролог/эпилог Go-функции



```
TEXT TestSumFunc(SB)
    MOVD 16(R28), R16
    CMP R16, RSP
    BLS 25(PC)
    . . .
```

```
func TestSumFunc(t *testing.T) {
    assert.Equal(t, 1, SumFunc(0, 1))
}
```

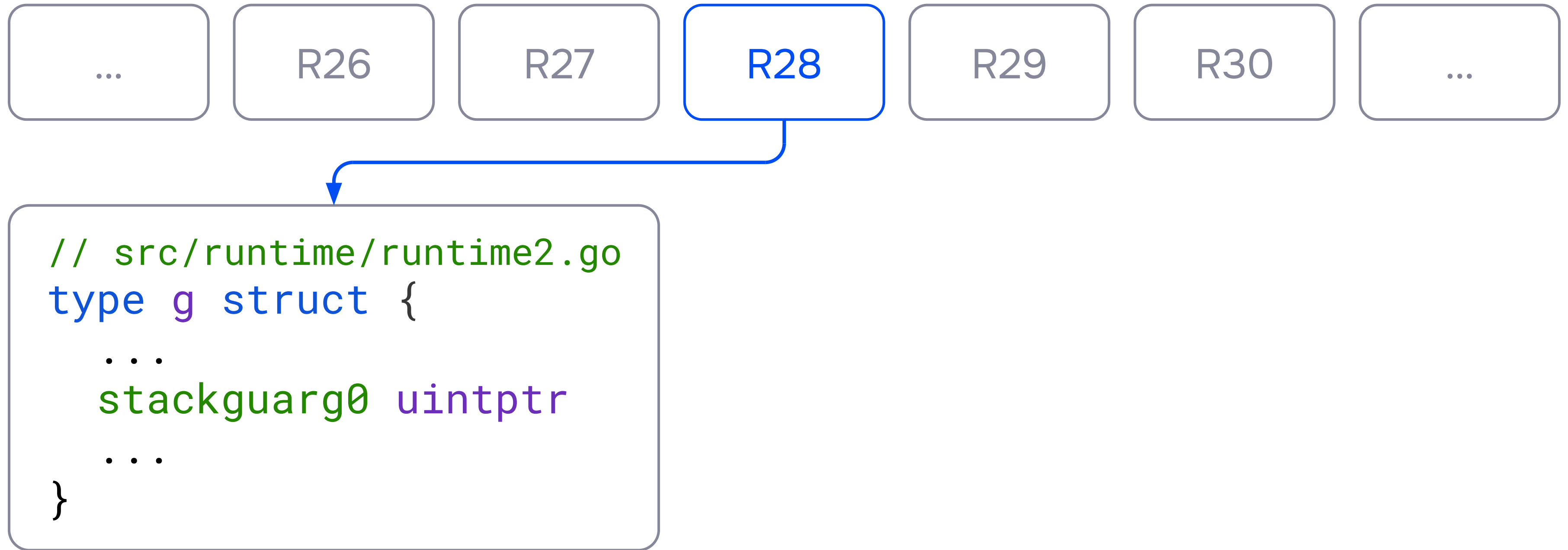
Пролог/эпилог Go-функции



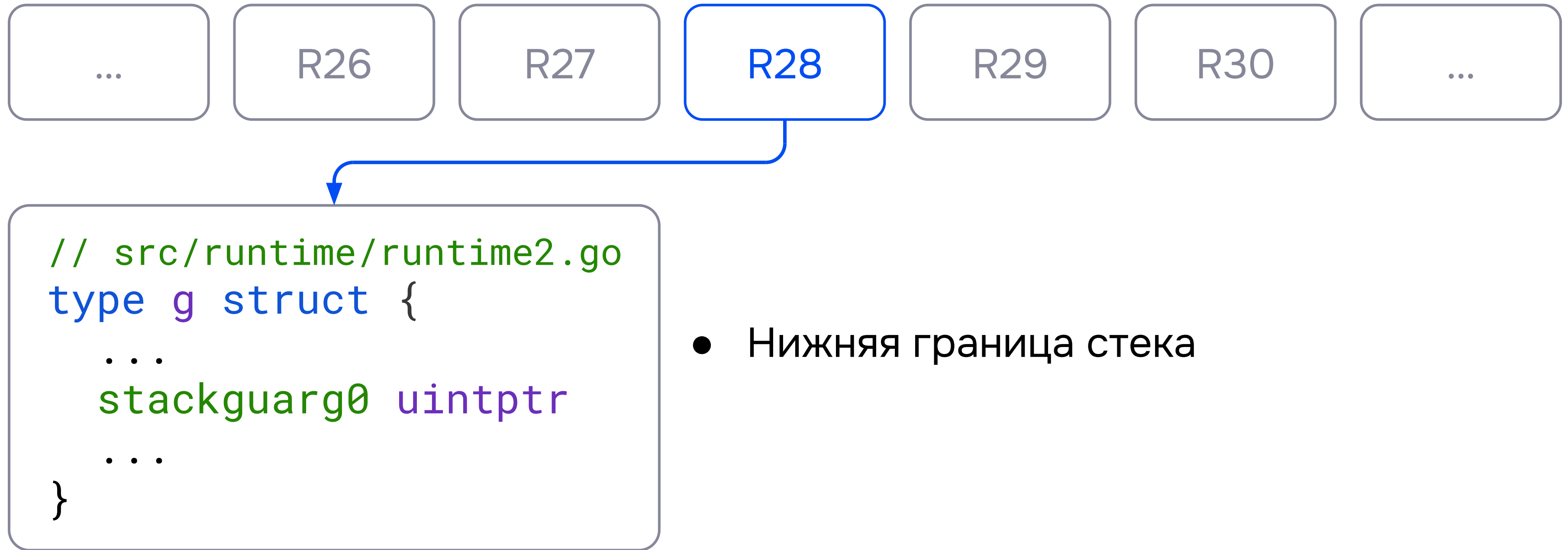
```
TEXT TestSumFunc(SB)
    MOVD 16(R28), R16
    CMP R16, RSP
    BLS 25(PC)
    . . .
```

```
func TestSumFunc(t *testing.T) {
    assert.Equal(t, 1, SumFunc(0, 1))
}
```

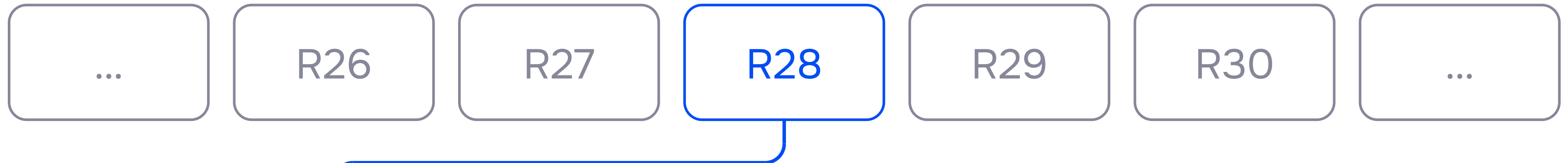
Пролог/эпилог Go-функции



Пролог/эпилог Go-функции



Пролог/эпилог Go-функции



```
// src/runtime/runtime2.go
type g struct {
    ...
    stackguard0 uintptr
    ...
}
```

- Нижняя граница стека
- Константа `0xFFFFFFFFFFFFFFFFFADE`

Пролог/эпилог Go-функции

```
TEXT TestSumFunc(SB)
    MOVD 16(R28), R16
    CMP R16, RSP
    BLS 25(PC)
    . . .
```



Осторожно
ASM

Пролог/эпилог Go-функции

```
TEXT TestSumFunc(SB)
    MOVD 16(R28), R16
    CMP R16, RSP
    BLS 25(PC)
    . . .
```



Осторожно
ASM

Пролог/эпилог Go-функции

```
TEXT TestSumFunc(SB)
    MOVD 16(R28), R16
    CMP R16, RSP
    BLS 25(PC)
    . . .
```



Осторожно
ASM

Пролог/эпилог Go-функции



```
TEXT TestSumFunc(SB)
    MOVD 16(R28), R16
    CMP R16, RSP
    BLS 25(PC)
    ...
    MOVD R0, 8(RSP)
    MOVD R30, R3
    CALL runtime.morestack_noctxt.abi0(SB)
    MOVD 8(RSP), R0
    JMP TestSumFunc(SB)
```

Пролог/эпилог Go-функции



```
TEXT TestSumFunc(SB)
    MOVD 16(R28), R16
    CMP R16, RSP
    BLS 25(PC)
    ...
    MOVD R0, 8(RSP)
    MOVD R30, R3
    CALL runtime.morestack_noctxt.abi0(SB)
    MOVD 8(RSP), R0
    JMP TestSumFunc(SB)
```

Пролог/эпилог Go-функции



```
TEXT TestSumFunc(SB)
    MOVD 16(R28), R16
    CMP R16, RSP
    BLS 25(PC)
    ...
    MOVD R0, 8(RSP)
    MOVD R30, R3
    CALL runtime.morestack_noctxt.abi0(SB)
    MOVD 8(RSP), R0
    JMP TestSumFunc(SB)
```

Пролог/эпилог Go-функции



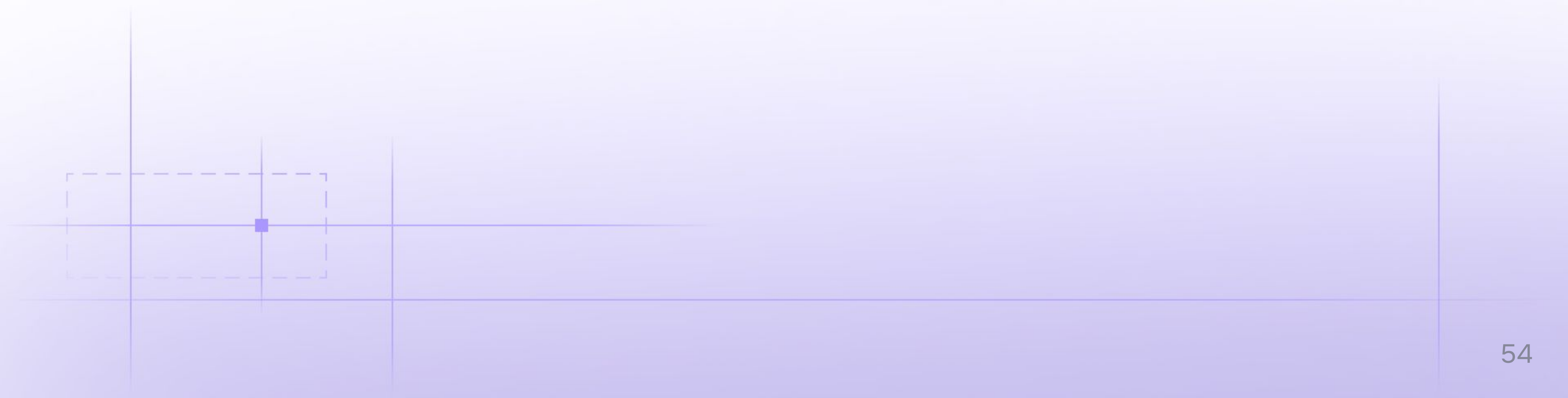
```
TEXT TestSumFunc(SB)
    MOVD 16(R28), R16
    CMP R16, RSP
    BLS 25(PC)
    ...
    MOVD R0, 8(RSP)
    MOVD R30, R3
    CALL runtime.morestack_noctxt.abi0(SB)
    MOVD 8(RSP), R0
    JMP TestSumFunc(SB)
```

Пролог/эпилог Go-функции



```
TEXT TestSumFunc(SB)
    MOVD 16(R28), R16
    CMP R16, RSP
    BLS 25(PC)
    ...
    MOVD R0, 8(RSP)
    MOVD R30, R3
    CALL runtime.morestack_noctxt.abi0(SB)
    MOVD 8(RSP), R0
    JMP TestSumFunc(SB)
```

Бенчмарк	Время	Дельта
Func	1.01 ns	+1.22 ns
FuncWithPrologue	2.23 ns	



Причины просадки производительности

- ✓ Дополнительные ASM-инструкции
- ✓ Чтение памяти структуры g
- ✓ Прوماхи кеша данных/инструкций

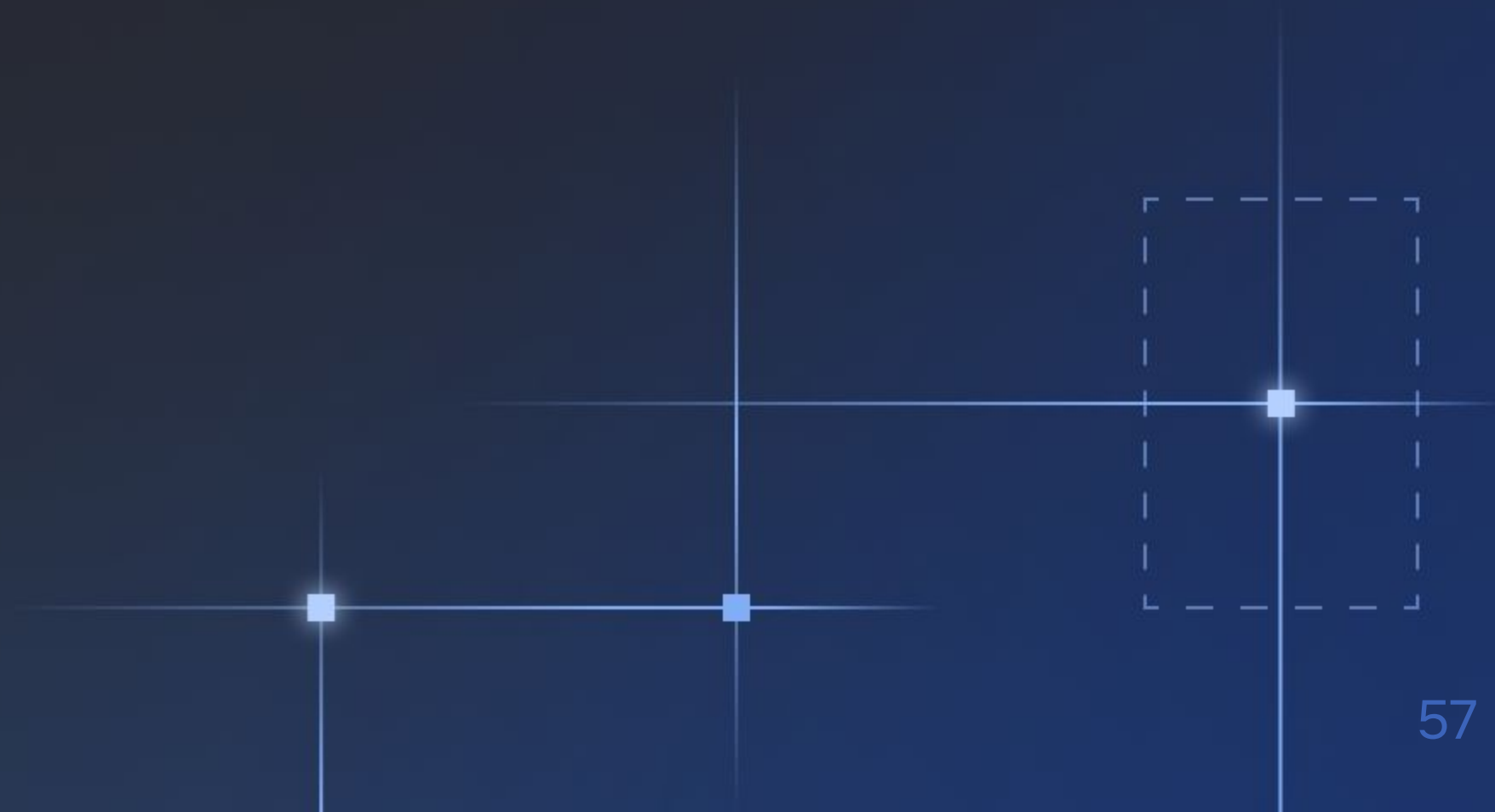


Советы по оптимизации

- ✓ Profile-Guided Optimization
- ✓ Ручной инлайнинг
(строго на основании бенчмарков!)



Интерфейсы



Интерфейсы

```
type Iterator interface {  
    Next() bool  
    At() int  
}
```

Интерфейсы

```
type IteratorImpl struct {  
    i      int  
    limit int  
}
```

Интерфейсы

```
type IteratorImpl struct {  
    i      int  
    limit  int  
}  
  
//go:noinline  
func (it *IteratorImpl) Next() bool {  
    if it.i < it.limit {  
        it.i++  
        return true  
    }  
  
    return false  
}
```

Интерфейсы

```
type IteratorImpl struct {  
    i      int  
    limit int  
}  
  
//go:noinline  
func (it *IteratorImpl) At() int {  
    return it.i  
}
```

Интерфейсы

```
//go:noinline
func IterateByFunc(it *IteratorImpl) int {
    result := 0
    for it.Next() {
        result += it.At()
    }
    return result
}
```

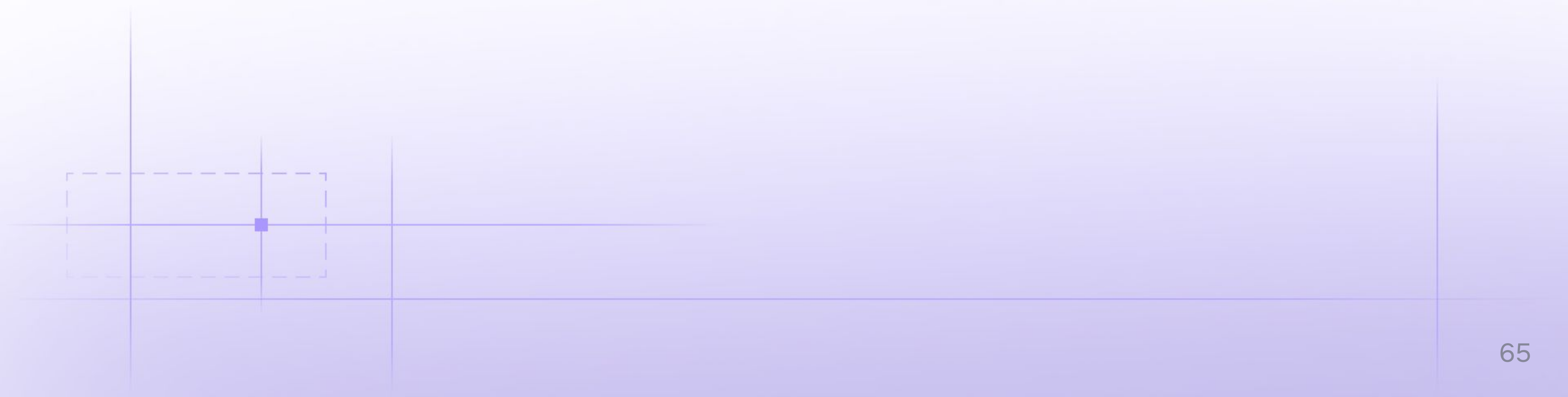
Интерфейсы

```
//go:noinline
func IterateByFunc(it *IteratorImpl) int {
    result := 0
    for it.Next() {
        result += it.At()
    }
    return result
}
```

Интерфейсы

```
//go:noinline
func IterateByInterface(it Iterator) int {
    result := 0
    for it.Next() {
        result += it.At()
    }
    return result
}
```

Бенчмарк	Кол-во итераций	Время	Дельта
FunctionCall	1	7.91 ns	+17% (+1.39 ns)
InterfaceCall	1	9.30 ns	



Бенчмарк	Кол-во итераций	Время	Дельта
FunctionCall	1	7.91 ns	+17% (+1.39 ns)
InterfaceCall	1	9.30 ns	
FunctionCall	10	24.33 ns	+27% (6.76 ns)
InterfaceCall	10	31.09 ns	



Бенчмарк	Кол-во итераций	Время	Дельта
FunctionCall	1	7.91 ns	+17% (+1.39 ns)
InterfaceCall	1	9.30 ns	
FunctionCall	10	24.33 ns	+27% (6.76 ns)
InterfaceCall	10	31.09 ns	
FunctionCall	100	224.01 ns	+15% (+35.02 ns)
InterfaceCall	100	259.03 ns	



Интерфейсы

```
//go:noinline  
func callNext(it Iterator) {  
    it.Next()  
}
```

Интерфейсы

```
//go:noinline
func callNext(it Iterator) {
    it.Next()
}

func iterate() {
    var it IteratorImpl
    callNext(&it)
}
```

Интерфейсы



```
TEXT iterate(SB)
```

```
...
```

```
MOVD $type:IteratorImpl(SB), R0
```

```
CALL runtime.newobject(SB)
```

```
MOVD R0, R1
```

```
MOVD $go:itab.*IteratorImpl,Iterator(SB), R0
```

```
CALL callNext(SB)
```

```
...
```

```
func iterate() {  
    var it IteratorImpl  
    callNext(&it)  
}
```

Интерфейсы

```
// src/runtime/malloc.go
func newobject(typ *_type) unsafe.Pointer {
    return mallocgc(typ.Size_, typ, true)
}
```

Интерфейсы

```
// src/internal/abi/type.go
type Type struct {
    Size_          uintptr
    PtrBytes       uintptr
    Hash           uint32
    TFlag          TFlag
    Align_         uint8
    FieldAlign_    uint8
    Kind_          Kind
    Equal func(unsafe.Pointer, unsafe.Pointer) bool
    GCData      *byte
    Str          NameOff
    PtrToThis    TypeOff
}
```

Интерфейсы



```
TEXT iterate(SB)
```

```
...
```

```
MOVD $type:IteratorImpl(SB), R0
```

```
CALL runtime.newobject(SB)
```

```
MOVD R0, R1
```

```
MOVD $go:itab.*IteratorImpl, Iterator(SB), R0
```

```
CALL callNext(SB)
```

```
...
```

```
func iterate() {  
    var it IteratorImpl  
    callNext(&it)  
}
```

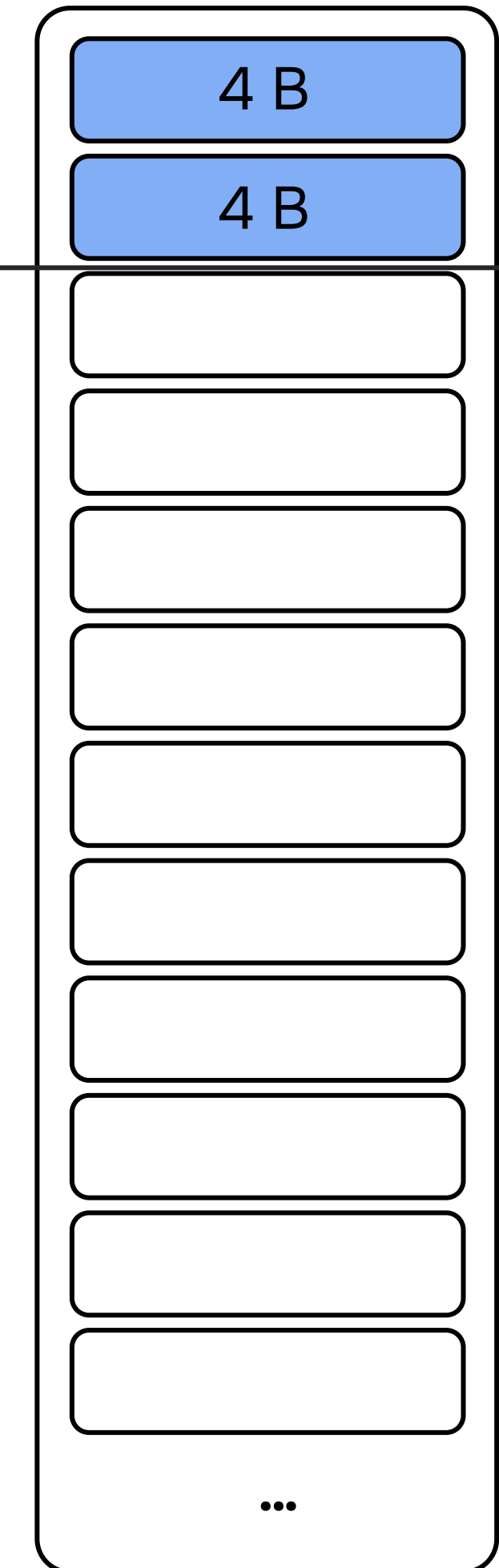
Интерфейсы

```
// src/internal/abi/iface.go
type ITab struct {
    Inter *InterfaceType
    Type  *Type
    Hash  uint32
    Fun   [1]uintptr
}
```

Интерфейсы

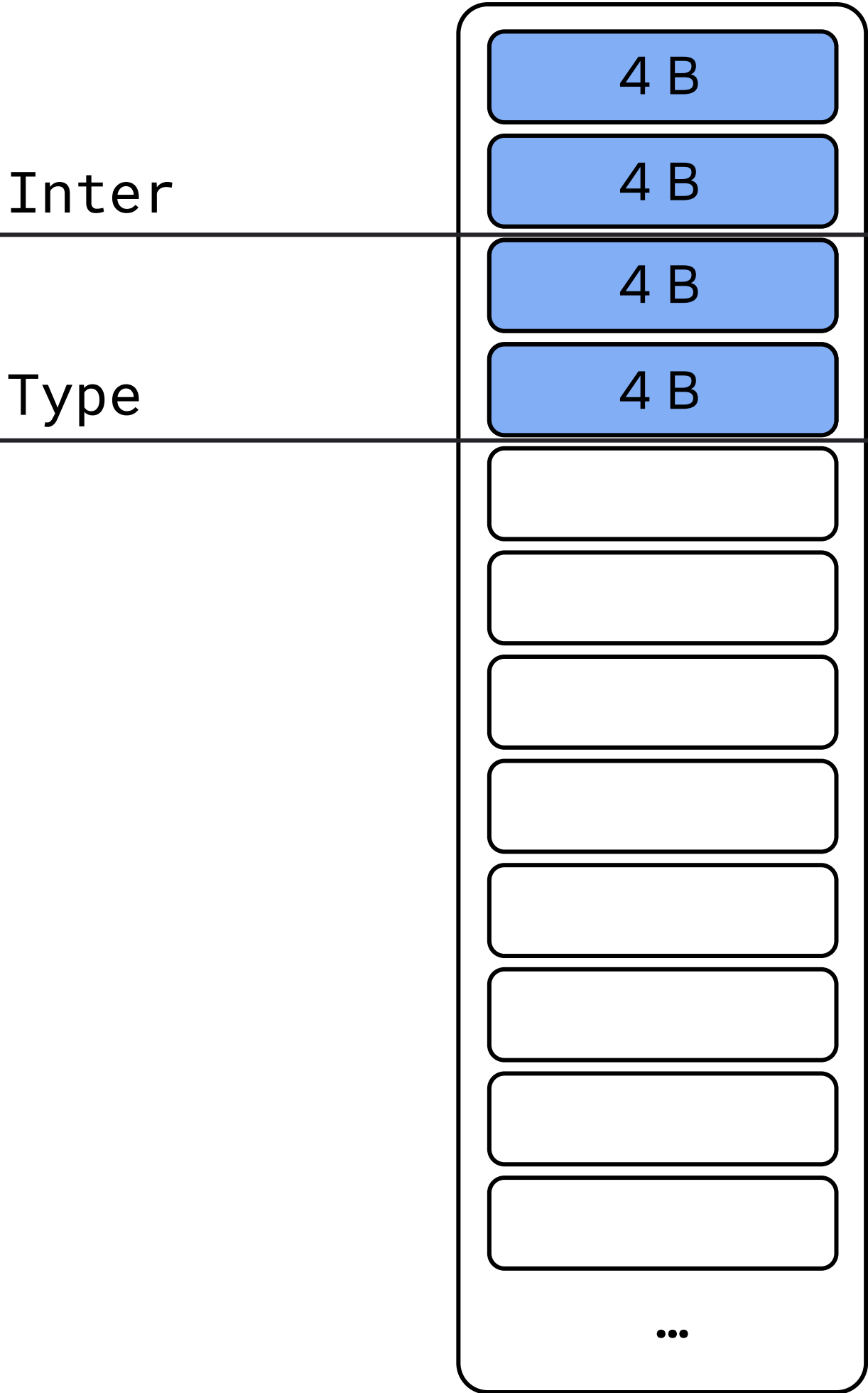
```
// src/internal/abi/iface.go
type ITab struct {
    Inter *InterfaceType
    Type  *Type
    Hash  uint32
    Fun   [1]uintptr
}
```

Inter



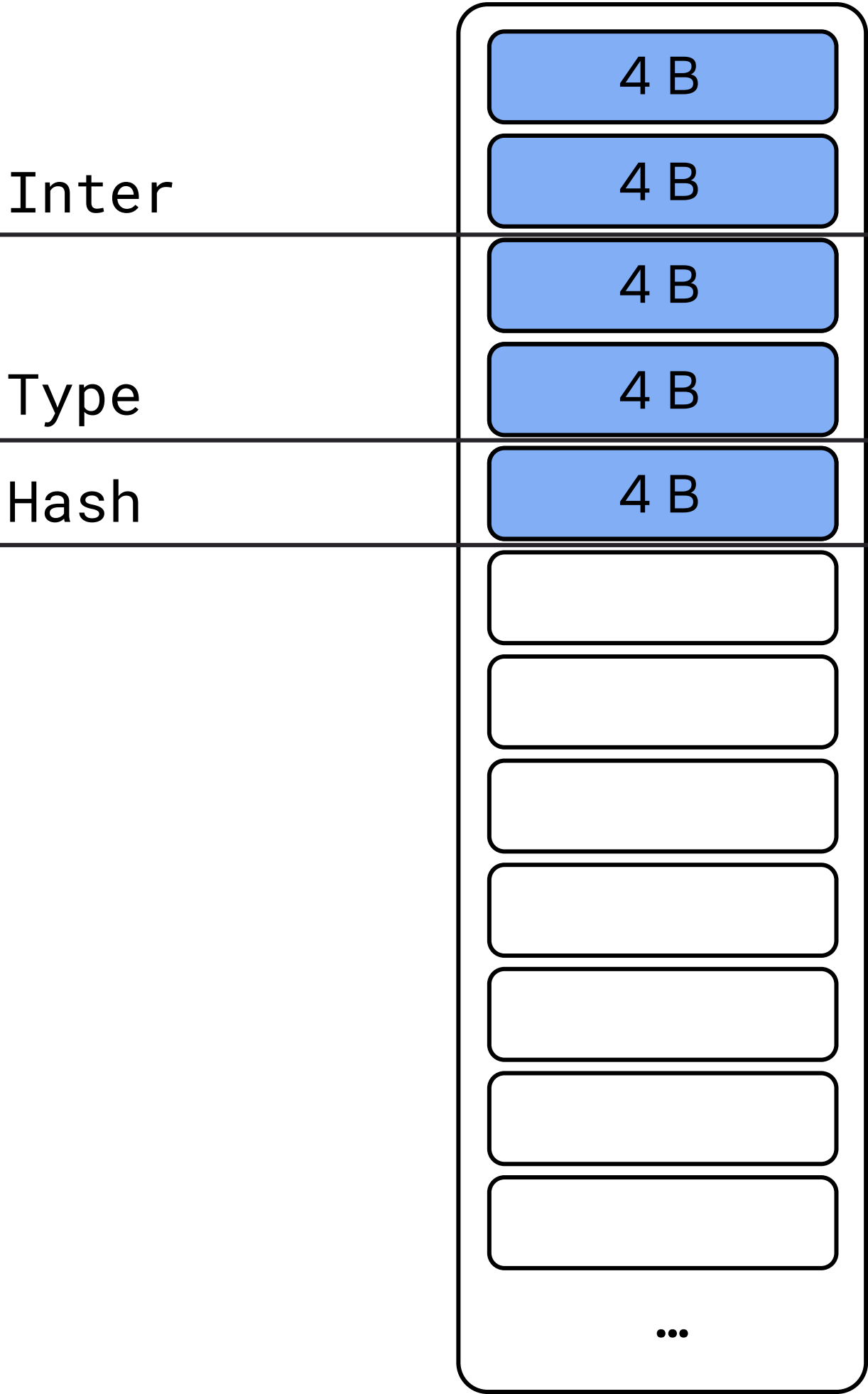
Интерфейсы

```
// src/internal/abi/iface.go
type ITab struct {
    Inter *InterfaceType
    Type  *Type
    Hash  uint32
    Fun   [1]uintptr
}
```



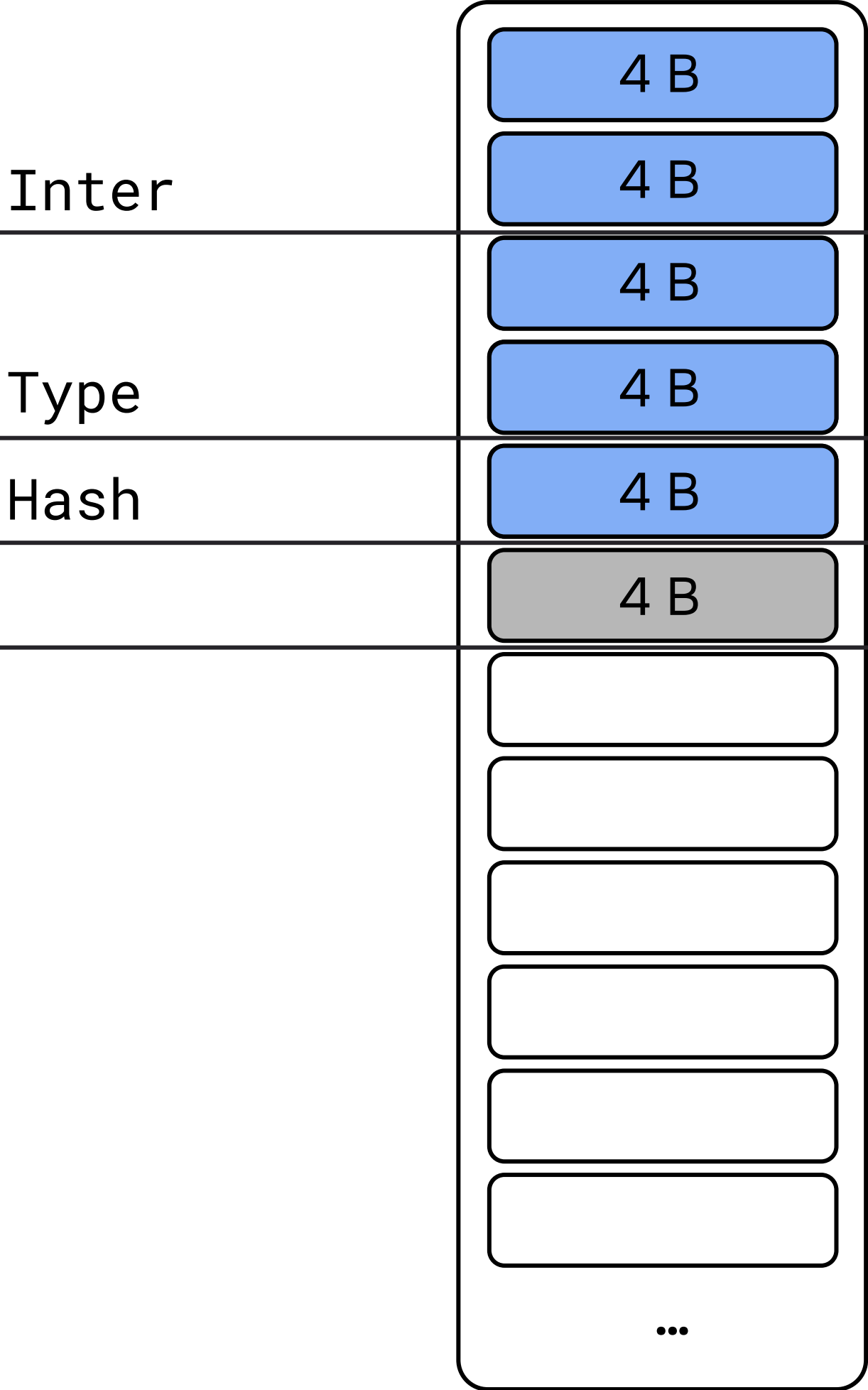
Интерфейсы

```
// src/internal/abi/iface.go
type ITab struct {
    Inter *InterfaceType
    Type  *Type
    Hash  uint32
    Fun   [1]uintptr
}
```



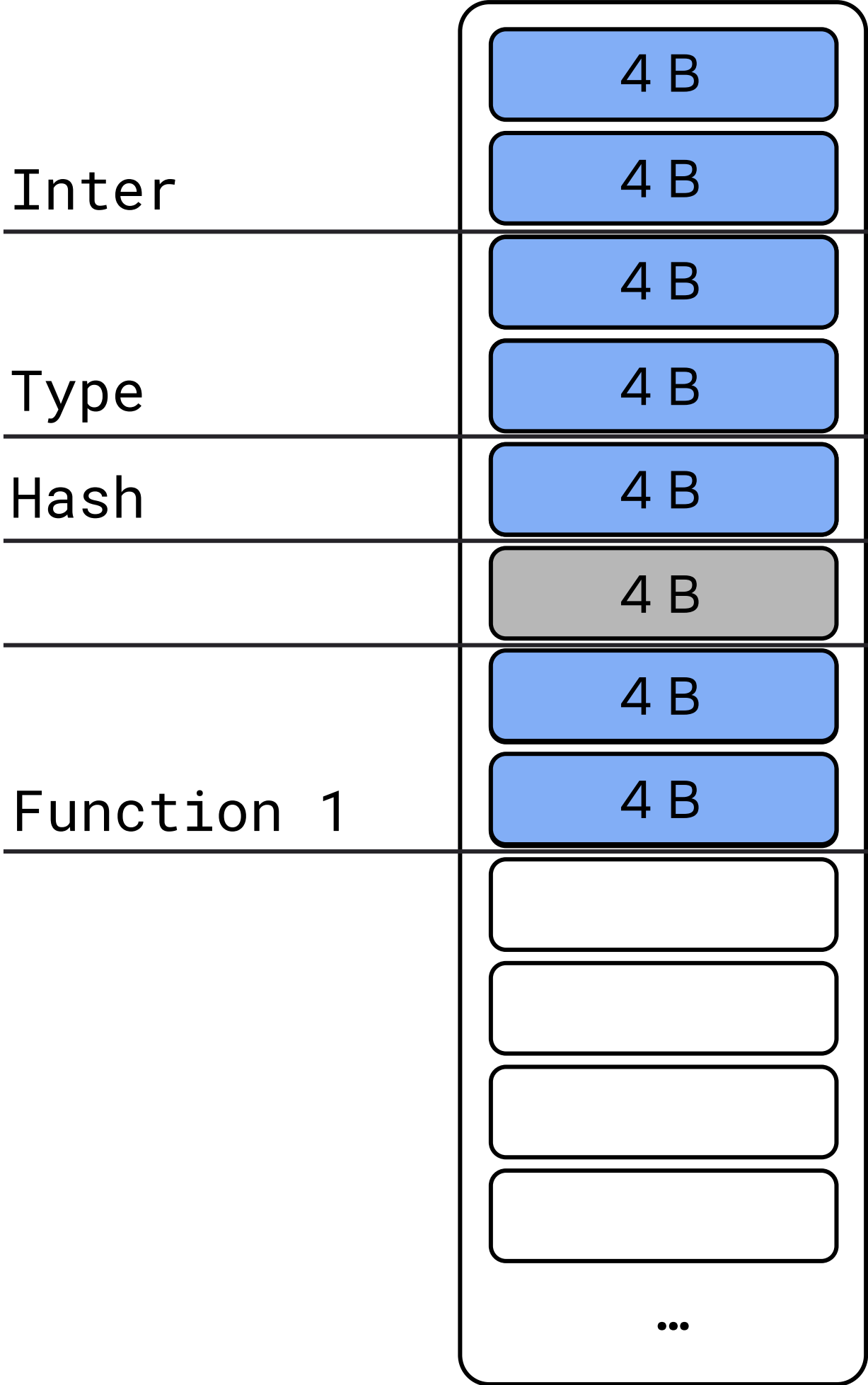
Интерфейсы

```
// src/internal/abi/iface.go
type ITab struct {
    Inter *InterfaceType
    Type  *Type
    Hash  uint32
    _     [4]byte // padding
    Fun   [1]uintptr
}
```



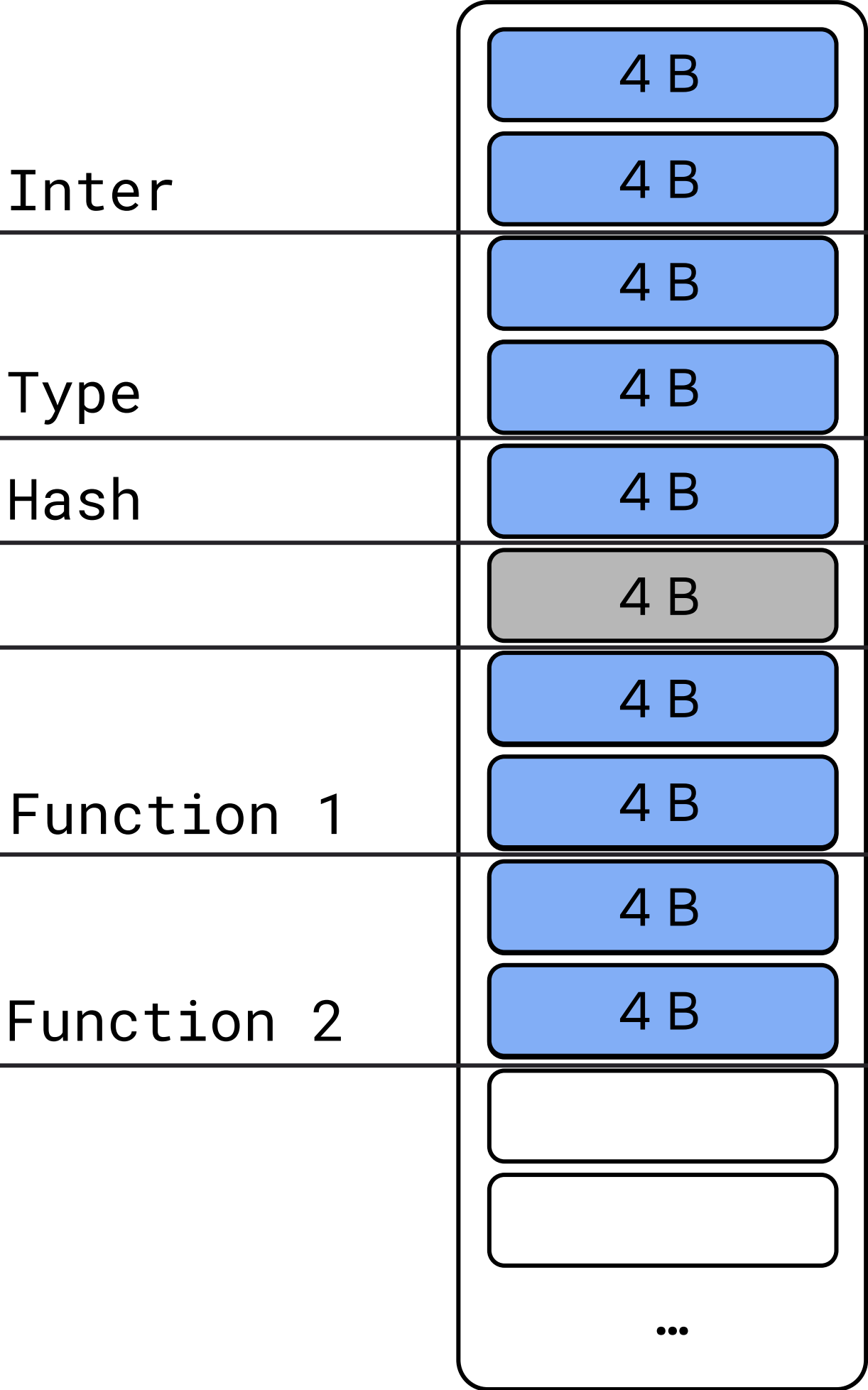
Интерфейсы

```
// src/internal/abi/iface.go
type ITab struct {
    Inter *InterfaceType
    Type  *Type
    Hash  uint32
    _     [4]byte // padding
    Fun   [1]uintptr
}
```



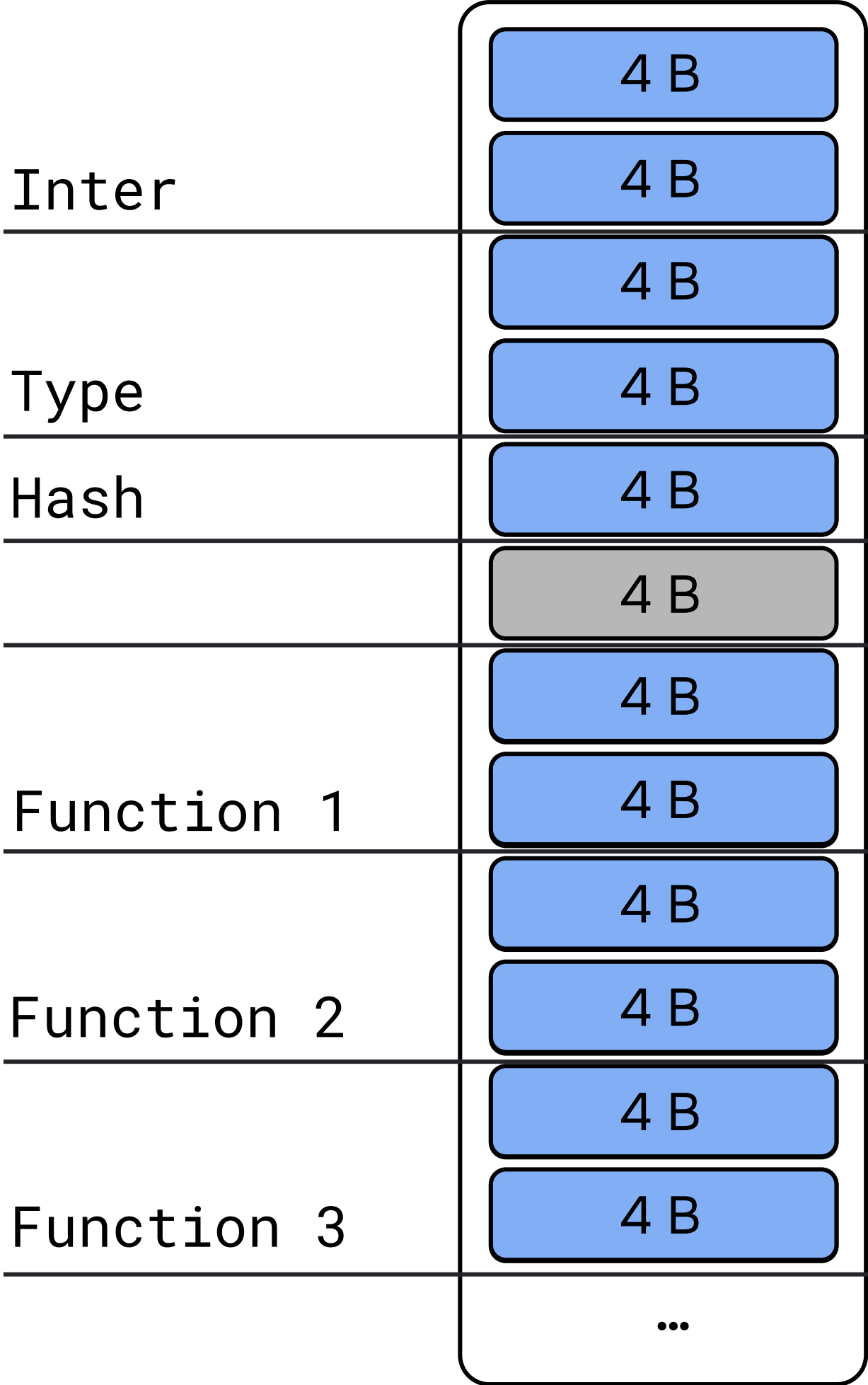
Интерфейсы

```
// src/internal/abi/iface.go
type ITab struct {
    Inter *InterfaceType
    Type  *Type
    Hash  uint32
    _     [4]byte // padding
    Fun   [1]uintptr
    ...
}
```



Интерфейсы

```
// src/internal/abi/iface.go
type ITab struct {
    Inter *InterfaceType
    Type  *Type
    Hash  uint32
    _     [4]byte // padding
    Fun   [1]uintptr
    ...
}
```



Интерфейсы

```
// src/internal/abi/iface.go
```

```
type ITab struct {  
    Inter *InterfaceType  
    Type  *Type  
    Hash  uint32  
    _     [4]byte // padding  
    Fun   []uintptr  
}
```

```
type Slice struct {  
    memory    *uintptr  
    size      uint64  
    capacity  uint64  
}
```

Интерфейсы

```
//go:noinline  
func callNext(it Iterator) {  
    it.Next()  
}
```

Интерфейсы

```
//go:noinline
func callNext(it Iterator) {
    it.Next()
}
```

```
TEXT callNext(SB)
    ...
    MOVD 32(R0), R2
    MOVD R1, R0
    CALL (R2)
    ...
```



Осторожно
ASM

Интерфейсы



```
//go:noinline
func callNext(it Iterator) {
    it.Next()
}
```

```
TEXT callNext(SB)
    ...
    MOVD 32(R0), R2
    MOVD R1, R0
    CALL (R2)
    ...
```

```
type ITab struct {
    Inter *InterfaceType
    Type  *Type
    Hash  uint32
    -     [4]byte // padding
    Fun   [1]uintptr
}
```

Интерфейсы

```
//go:noinline
func callNext(it Iterator) {
    it.Next()
}
```

```
TEXT callNext(SB)
    ...
    MOVD 32(R0), R2
    MOVD R1, R0
    CALL (R2)
    ...
```



Осторожно
ASM

Причины просадки производительности

- ✓ Интерфейсы скрывают информацию о потоке выполнения от компилятора, заставляя генерировать обобщённый код



Интерфейсы

```
//go:noinline
func callNext(it Iterator) {
    i, _ := it.(*IteratorImpl)
    i.Next()
}
```

Интерфейсы



```
//go:noinline
func callNext(it Iterator) {
    i, _ := it.(*IteratorImpl)
    i.Next()
}
```

```
TEXT callNext(SB)
    . . .
    MOVD $go:itab.*IteratorImpl, Iterator(SB), R2
    CMP R2, R0
    . . .
    MOVD R1, R0
    CALL (*IteratorImpl).Next(SB)
    . . .
```

```
//go:noinline
func IterateByInterfaceBySwitch(it Iterator) int {
    result := 0

    switch v := it.(type) {
    case *IteratorImpl:
        for v.Next() {
            result += v.At()
        }

    default:
        for v.Next() {
            result += v.At()
        }
    }

    return result
}
```

```
//go:noinline
func IterateByInterfaceBySwitch(it Iterator) int {
    result := 0

    switch v := it.(type) {
    case *IteratorImpl:
        for v.Next() {
            result += v.At()
        }

    default:
        for v.Next() {
            result += v.At()
        }
    }

    return result
}
```

```
//go:noinline
func IterateByInterfaceBySwitch(it Iterator) int {
    result := 0

    switch v := it.(type) {
    case *IteratorImpl:
        for v.Next() {
            result += v.At()
        }

    default:
        for v.Next() {
            result += v.At()
        }
    }

    return result
}
```

```
//go:noinline
func IterateByInterfaceBySwitch(it Iterator) int {
    result := 0

    switch v := it.(type) {
    case *IteratorImpl:
        for v.Next() {
            result += v.At()
        }

    default:
        for v.Next() {
            result += v.At()
        }
    }

    return result
}
```

Бенчмарк	Кол-во итераций	Время	Дельта
InterfaceCall	1	9.30 ns	+7% (+0.68 ns)
InterfaceCallBySwitch	1	9.98 ns	



Бенчмарк	Кол-во итераций	Время	Дельта
InterfaceCall	1	9.30 ns	+7% (+0.68 ns)
InterfaceCallBySwitch	1	9.98 ns	
InterfaceCall	10	31.09 ns	-10% (-3.13 ns)
InterfaceCallBySwitch	10	27.96 ns	



Бенчмарк	Кол-во итераций	Время	Дельта
InterfaceCall	1	9.30 ns	+7% (+0.68 ns)
InterfaceCallBySwitch	1	9.98 ns	
InterfaceCall	10	31.09 ns	-10% (-3.13 ns)
InterfaceCallBySwitch	10	27.96 ns	
InterfaceCall	100	259.03 ns	-11% (-30.23 ns)
InterfaceCallBySwitch	100	228.80 ns	



Интерфейсы

```
//go:noinline
func (it *ChunkIterator) Next() bool {
    if it.i < it.limit {
        it.i++
        return true
    }

    return false
}

//go:noinline
func (it *IteratorImpl) At() int {
    return it.i
}
```

Интерфейсы

```
func (it *ChunkIterator) Next() bool {  
    if it.i < it.limit {  
        it.i++  
        return true  
    }
```

```
    return false  
}
```

```
func (it *IteratorImpl) At() int {  
    return it.i  
}
```

Бенчмарк	Кол-во итераций	Время	Дельта
InterfaceCall	1	9.30 ns	-37% (-3.45 ns)
InterfaceCallBySwitch	1	5.85 ns	



Бенчмарк	Кол-во итераций	Время	Дельта
InterfaceCall	1	9.30 ns	-37% (-3.45 ns)
InterfaceCallBySwitch	1	5.85 ns	
InterfaceCall	10	31.09 ns	-30% (-9.61 ns)
InterfaceCallBySwitch	10	21.48 ns	



Бенчмарк	Кол-во итераций	Время	Дельта
InterfaceCall	1	9.30 ns	-37% (-3.45 ns)
InterfaceCallBySwitch	1	5.85 ns	
InterfaceCall	10	31.09 ns	-30% (-9.61 ns)
InterfaceCallBySwitch	10	21.48 ns	
InterfaceCall	100	259.03 ns	-16% (-43.03 ns)
InterfaceCallBySwitch	100	216.00 ns	



Бенчмарк	Кол-во итераций	Время	Дельта
FunctionCall	1	4.12 ns	-41 % (-1.73 ns)
InterfaceCallBySwitch	1	5.85 ns	
FunctionCall	10	19.49 ns	-10 % (-1.99 ns)
InterfaceCallBySwitch	10	21.48 ns	
FunctionCall	100	203.00 ns	-6 % (-13.00 ns)
InterfaceCallBySwitch	100	216.00 ns	



Советы по оптимизации

- ✓ Необходимо создать подсказки для компилятора



Range over function



Range over function

```
type Sample struct {  
    timestamp int64  
    value float64  
}
```

Range over function

```
type Sample struct {  
    timestamp int64  
    value float64  
}  
  
type SampleStorage interface {  
    GetSamples() []Sample  
}
```

Range over function

```
type Sample struct {  
    timestamp int64  
    value float64  
}  
  
type SampleStorage interface {  
    GetSamples() []Sample  
    Samples(func (Sample) bool)  
}
```

Range over function

```
type Sample struct {  
    timestamp int64  
    value float64  
}  
  
type SampleStorage interface {  
    GetSamples() []Sample  
    Samples(func (Sample) bool)  
}  
  
for s := range h.Samples {  
    ...  
}
```

Range over function

```
type SampleStorageImpl struct {  
    samples []Sample  
}
```

Range over function

```
type SampleStorageImpl struct {  
    samples []Sample  
}  
  
func (s *SampleStorageImpl) GetSamples() []Sample {  
    return s.samples  
}
```

Range over function

```
type SampleStorageImpl struct {
    samples []Sample
}

func (s *SampleStorageImpl) GetSamples() []Sample {
    return s.samples
}

func (s *SampleStorageImpl) Samples(f func(Sample) bool) {
    for i := range s.samples {
        if !f(s.samples[i]) {
            break
        }
    }
}
```

Range over function

```
func BenchmarkSlice(b *testing.B) {  
    s := NewSampleStorageImpl(samplesCount)  
    sum := int64(0)  
  
    for b.Loop() {  
        for _, s := range s.GetSamples() {  
            sum += s.timestamp  
        }  
    }  
}
```

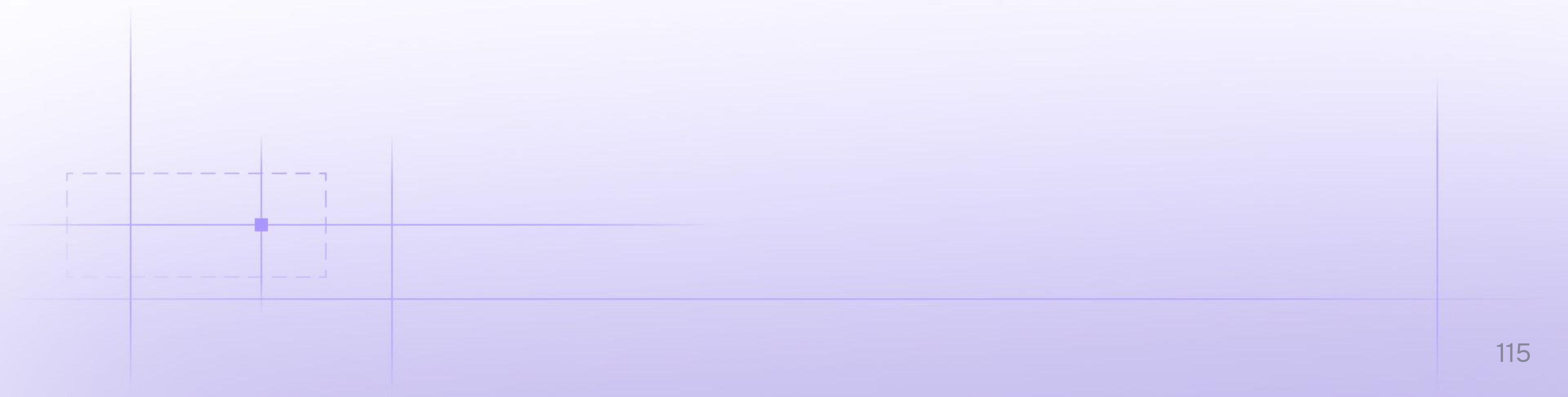
Range over function

```
func BenchmarkSlice(b *testing.B) {  
    s := NewSampleStorageImpl(samplesCount)  
    sum := int64(0)  
  
    for b.Loop() {  
        for _, s := range s.GetSamples() {  
            sum += s.timestamp  
        }  
    }  
}
```

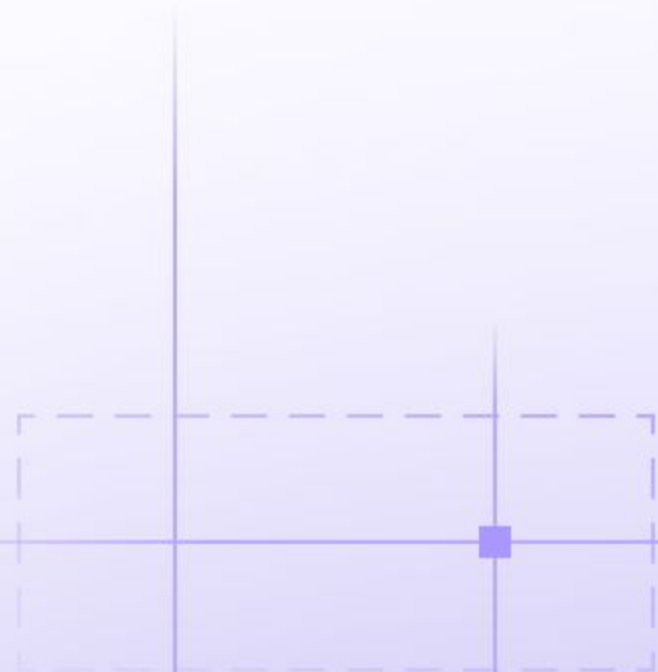
Range over function

```
func BenchmarkRangeOverFunc(b *testing.B) {  
    s := NewSampleStorageImpl(samplesCount)  
    sum := int64(0)  
  
    for b.Loop() {  
        for sample := range s.Samples {  
            sum += sample.timestamp  
        }  
    }  
}
```

Бенчмарк	Кол-во элементов	Время	Дельта
Slice	1	2.07 ns	+226 % (+4.68 ns)
RangeOverFunc	1	6.75 ns	



Бенчмарк	Кол-во элементов	Время	Дельта
Slice	1	2.07 ns	+226 % (+4.68 ns)
RangeOverFunc	1	6.75 ns	
Slice	10	18.17 ns	+146 % (+26.69 ns)
RangeOverFunc	10	44.86 ns	



Бенчмарк	Кол-во элементов	Время	Дельта
Slice	1	2.07 ns	+226 % (+4.68 ns)
RangeOverFunc	1	6.75 ns	
Slice	10	18.17 ns	+146 % (+26.69 ns)
RangeOverFunc	10	44.86 ns	
Slice	100	182.20 ns	+140 % (+256.00 ns)
RangeOverFunc	100	438.20 ns	



***просчитался,
но...
где?***



Range over function

```
func (s *SampleStorageImpl) Samples(f func(Sample) bool) {  
    for i := range s.samples {  
        if !f(s.samples[i]) {  
            break  
        }  
    }  
}
```

Range over function

```
func BenchmarkRangeOverFunc(b *testing.B) {  
    s := NewSampleStorageImpl(samplesCount)  
    sum := int64(0)  
  
    for b.Loop() {  
        for sample := range s.Samples {  
            sum += sample.timestamp  
        }  
    }  
}
```

Range over function

```
func BenchmarkRangeOverFunc(b *testing.B) {  
    s := NewSampleStorageImpl(samplesCount)  
    sum := int64(0)  
  
    for b.Loop() {  
        for sample := range s.Samples {  
            sum += sample.timestamp  
        }  
    }  
}
```

Range over function



```
TEXT BenchmarkRangeOverFunc(SB)
```

```
...  
MOVD ZR, sum-1712(SP)  
MOVD $1, R2  
MOVD R2, #state1-64(SP)  
MOVD $BenchmarkRange-range1(SB), R2  
MOVD R2, .autotmp_19-32(SP)  
MOVD $#state1-64(SP), R2  
MOVD $sum-1712(SP), R3  
STP (R2, R3), .autotmp_19-24(SP)  
MOVD $.autotmp_19-32(SP), R2  
MOVD R2, #yield1-8(SP)  
MOVD #yield1-8(SP), R1  
MOVD $s-56(SP), R0  
CALL command-line-arguments.(*SampleStorageImpl).Samples(SB)  
...
```

Range over function

```
type rangeClosure struct {  
    fn      uintptr  
    state *int  
    sum     *int64  
}
```

Range over function

```
type rangeClosure struct {  
    fn      uintptr  
    state   *int  
    sum     *int64  
}
```

Range over function

```
type rangeClosure struct {  
    fn      uintptr  
    state *int  
    sum     *int64  
}
```

Range over function

```
type rangeClosure struct {  
    fn      uintptr  
    state *int  
    sum     *int64  
}
```

Range over function



```
TEXT (*SampleStorageImpl).Samples(SB)
```

```
...  
MOVD    (R1), R2
```

```
...  
MOVD    R1, R26  
CALL    (R2)
```

```
...
```

```
func (s *SampleStorageImpl) Samples(f func(Sample) bool) {  
    for i := range s.samples {  
        if !f(s.samples[i]) {  
            break  
        }  
    }  
}
```

Range over function



```
TEXT (*SampleStorageImpl).Samples(SB)
```

```
...  
MOVD    (R1), R2
```

```
...  
MOVD    R1, R26  
CALL    (R2)
```

```
...
```

```
func (s *SampleStorageImpl) Samples(f func(Sample) bool) {  
    for i := range s.samples {  
        if !f(s.samples[i]) {  
            break  
        }  
    }  
}
```

TEXT BenchmarkRangeOverFunc-range1(SB)

```
...  
LDP 8(R26), (R1, R2)  
MOVD (R1), R3  
MOVD $2, R4  
MOVD R4, (R1)  
CMP $1, R3  
BNE 84  
...
```

```
func BenchmarkRangeOverFunc(b *testing.B) {  
    ...  
    for b.Loop() {  
        for sample := range s.Samples {  
            sum += sample.timestamp  
        }  
    }  
}
```



TEXT BenchmarkRangeOverFunc-range1(SB)

```
...  
LDP 8(R26), (R1, R2)  
MOVD (R1), R3  
MOVD $2, R4  
MOVD R4, (R1)  
CMP $1, R3  
BNE 84
```

```
...  
CALL runtime.panicrangestate(SB)
```

```
// Go  
currentState := *rangeClosure.state  
*rangeClosure.state = 2  
if currentState != 1 {  
    runtime.panicrangestate()  
}
```



TEXT BenchmarkRangeOverFunc-range1(SB)

```
...  
LDP 8(R26), (R1, R2)  
MOVD (R1), R3  
MOVD $2, R4  
MOVD R4, (R1)  
CMP $1, R3  
BNE 84
```

```
...  
CALL runtime.panicrangestate(SB)
```

```
// Go  
currentState := *rangeClosure.state  
*rangeClosure.state = 2  
if currentState != 1 {  
    runtime.panicrangestate()  
}
```



TEXT BenchmarkRangeOverFunc-range1(SB)

...

MOVD \$1, R2

MOVD R2, (R1)

...

// Go

*rangeClosure.state = 1



Причины просадки производительности

- ✓ Лямбда плохо инлайнится
- ✓ Проверка/изменение range-состояния в переменной **state** на каждом шаге цикла
- ✓ Работа с указателями на переменные в контексте замыкания (2 прыжка по памяти)



Range over function

```
func BenchmarkLambdaRange(b *testing.B) {  
    s := NewSampleStorageImpl(samplesCount)  
    sum := int64(0)  
  
    for b.Loop() {  
        for sample := range s.Samples {  
            sum += sample.timestamp  
        }  
    }  
}
```

Range over function

```
func BenchmarkLambdaRange(b *testing.B) {  
    s := NewSampleStorageImpl(samplesCount)  
    sum := int64(0)  
  
    for b.Loop() {  
        s.Samples(func(sample Sample) bool {  
            sum += sample.timestamp  
            return true  
        })  
    }  
}
```

Бенчмарк	Кол-во элементов	Время	Дельта
RangeOverFunc	1	6.75 ns	-18 % (-1.26 ns)
LambdaRange	1	5.49 ns	



Бенчмарк	Кол-во элементов	Время	Дельта
RangeOverFunc	1	6.75 ns	-18 % (-1.26 ns)
LambdaRange	1	5.49 ns	
RangeOverFunc	10	44.86 ns	-38 % (-17.37 ns)
LambdaRange	10	27.49 ns	



Бенчмарк	Кол-во элементов	Время	Дельта
RangeOverFunc	1	6.75 ns	-18 % (-1.26 ns)
LambdaRange	1	5.49 ns	
RangeOverFunc	10	44.86 ns	-38 % (-17.37 ns)
LambdaRange	10	27.49 ns	
RangeOverFunc	100	438.20 ns	-39 % (-173.10 ns)
LambdaRange	100	265.10 ns	

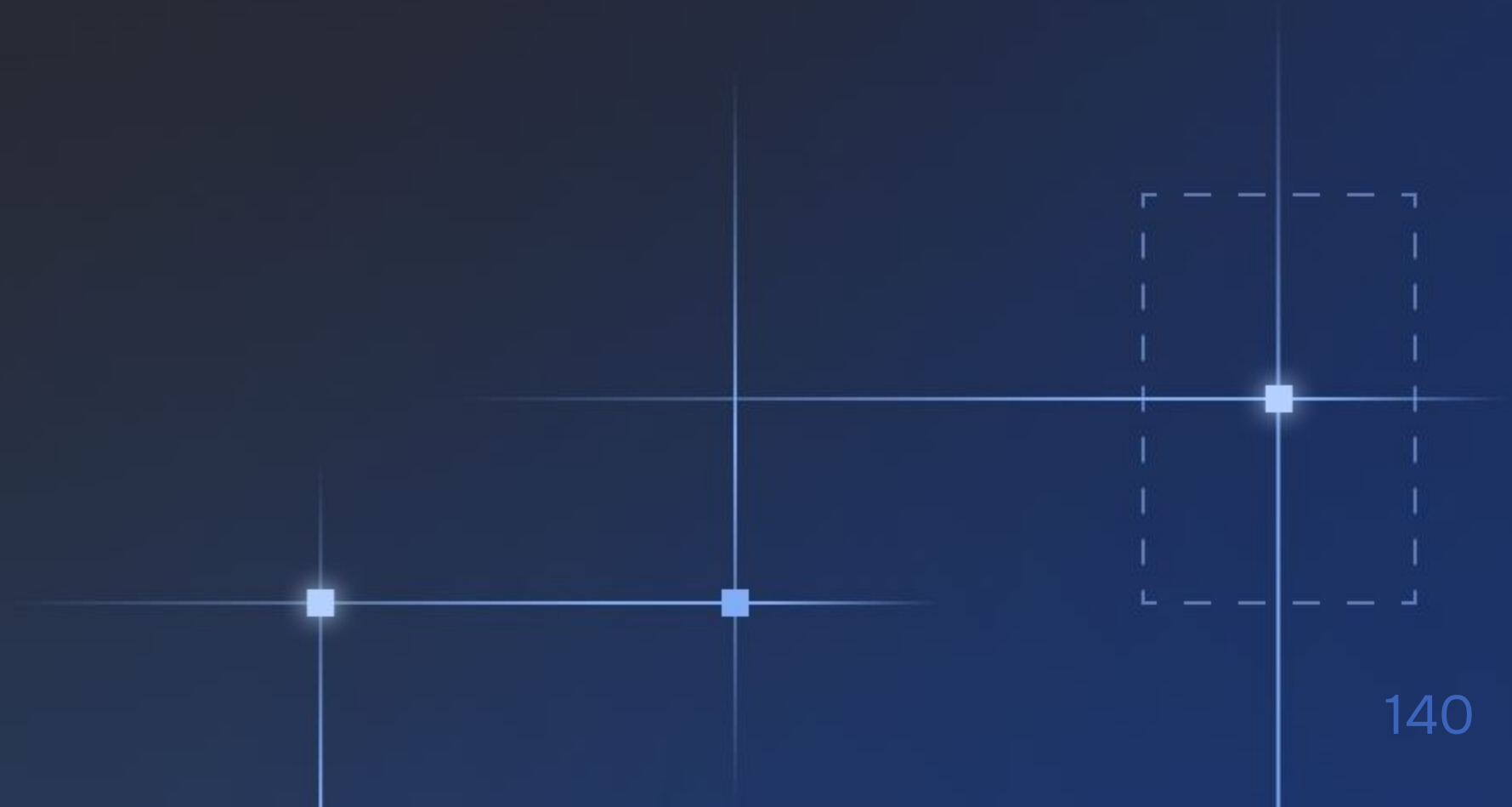


Советы по оптимизации

- ✓ Отказаться от синтаксического «сахара» в пользу менее читабельного, но производительного кода
- ✓ Бенчмарки, бенчмарки и ещё раз бенчмарки!



Defer



Defer

```
//go:noinline
func sumThroughFunc(from int, numbers []int) int {
    sum := 0
    for ; from < len(numbers); from++ {
        sum += numbers[from]
    }
    return sum
}
```

Defer

```
func sumThroughDefer(from int, numbers []int) (res int) {  
    defer func() {  
        for ; from < len(numbers); from++ {  
            res += numbers[from]  
        }  
    }()  
  
    return  
}
```

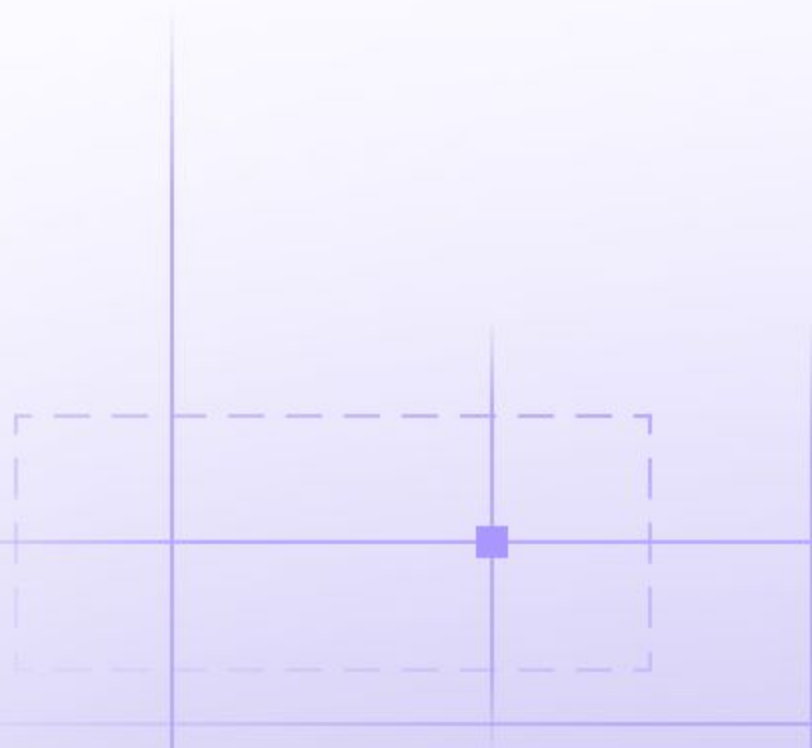
Defer

```
func sumThroughDefer(from int, numbers []int) (res int) {  
    defer func() {  
        for ; from < len(numbers); from++ {  
            res += numbers[from]  
        }  
    }()  
  
    return  
}
```

Defer

```
func sumThroughDefer(from int, numbers []int) (res int) {  
    defer func() {  
        for ; from < len(numbers); from++ {  
            res += numbers[from]  
        }  
    }()  
  
    return  
}
```

Бенчмарк	Кол-во итераций	Время	Дельта
SumThroughFunc	1	4.86 ns	+107% (+5.21 ns)
SumThroughDefer	1	10.07 ns	



Бенчмарк	Кол-во итераций	Время	Дельта
SumThroughFunc	1	4.86 ns	+107% (+5.21 ns)
SumThroughDefer	1	10.07 ns	
SumThroughFunc	10	10.68 ns	+169% (+18.10 ns)
SumThroughDefer	10	28.78 ns	



Бенчмарк	Кол-во итераций	Время	Дельта
SumThroughFunc	1	4.86 ns	+107% (+5.21 ns)
SumThroughDefer	1	10.07 ns	
SumThroughFunc	10	10.68 ns	+169% (+18.10 ns)
SumThroughDefer	10	28.78 ns	
SumThroughFunc	100	73.48 ns	+211% (+155.62 ns)
SumThroughDefer	100	229.10 ns	



Defer

```
func sumThroughDefer(from int, numbers []int) (res int) {  
    defer func() {  
        for ; from < len(numbers); from++ {  
            res += numbers[from]  
        }  
    }()  
  
    return 0  
}
```

```
TEXT sumThroughDefer(SB)
```

```
...
```

```
MOVD $sumThroughDefer.func1(SB), R0
```

```
MOVD $from(FP), R4
```

```
STP (R0, R4), .autotmp_4-56(SP)
```

```
STP (R1, R2), .autotmp_4-40(SP)
```

```
MOVD $res-64(SP), R0
```

```
STP (R3, R0), .autotmp_4-24(SP)
```

```
...
```



Осторожно
ASM

```
TEXT sumThroughDefer(SB)
```

```
...  
MOVD $sumThroughDefer.func1(SB), R0  
MOVD $from(FP), R4  
STP (R0, R4), .autotmp_4-56(SP)  
STP (R1, R2), .autotmp_4-40(SP)  
MOVD $res-64(SP), R0  
STP (R3, R0), .autotmp_4-24(SP)  
...
```

```
type deferClosure struct {  
    deferFn uintptr  
    from    *int  
    numbers []int  
    res     *int  
}
```



Defer



```
TEXT sumThroughDefer(SB)
```

```
...
```

```
MOVD R26, .autotmp_6-8(SP) // defer1 closure context
```

```
...
```

```
MOVB ZR, .autotmp_5-65(SP) // active defer bitset
```

```
...
```

Defer



```
TEXT sumThroughDefer(SB)
```

```
...
```

```
MOVD R26, .autotmp_6-8(SP) // defer1 closure context
```

```
...
```

```
MOVB ZR, .autotmp_5-65(SP) // active defers bitset
```

```
...
```

Defer

```
TEXT sumThroughDefer(SB)
    ...
    MOVD $.autotmp_4-56(SP), R26
    ...
    CALL (R0)
    ...
```



Осторожно
ASM

Defer

```
func sumThroughDefer(from int, numbers []int) (res int) {  
    defer func() {  
        for ; from < len(numbers); from++ {  
            res += numbers[from]  
        }  
    }()  
  
    return 0  
}
```

Defer

```
func sumThroughDefer(from int, numbers []int) (res int) {  
    defer func() {  
        for ; from < len(numbers); from++ {  
            res += numbers[from]  
        }  
    }()  
  
    return 0  
}
```

```
type deferClosure struct {  
    deferFn uintptr  
    from    *int  
    numbers []int  
    res     *int  
}
```

Defer

```
func sumThroughDefer(from int, numbers []int) (res int) {  
    defer func() {  
        for ; from < len(numbers); from++ {  
            res += numbers[from]  
        }  
    }()  
  
    return 0  
}
```

```
type deferClosure struct {  
    deferFn uintptr  
    from    *int  
    numbers []int  
    res     *int  
}
```

Defer

```
func sumThroughDefer(from int, numbers []int) (res int) {  
    defer func() {  
        sum := 0  
        for f := from; f < len(numbers); f++ {  
            sum += numbers[f]  
        }  
        res = sum  
    }()  
  
    return 0  
}
```

Defer

```
func sumThroughDefer(from int, numbers []int) (res int) {  
    defer func() {  
        sum := 0  
        for f := from; f < len(numbers); f++ {  
            sum += numbers[f]  
        }  
        res = sum  
    }()  
  
    return 0  
}
```

Defer

```
func sumThroughDefer(from int, numbers []int) (res int) {  
    defer func() {  
        sum := 0  
        for f := from; f < len(numbers); f++ {  
            sum += numbers[f]  
        }  
        res = sum  
    }()  
  
    return 0  
}
```

Defer

```
func sumThroughDefer(from int, numbers []int) (res int) {  
    defer func() {  
        sum := 0  
        for f := from; f < len(numbers); f++ {  
            sum += numbers[f]  
        }  
        res = sum  
    }()  
  
    return 0  
}
```

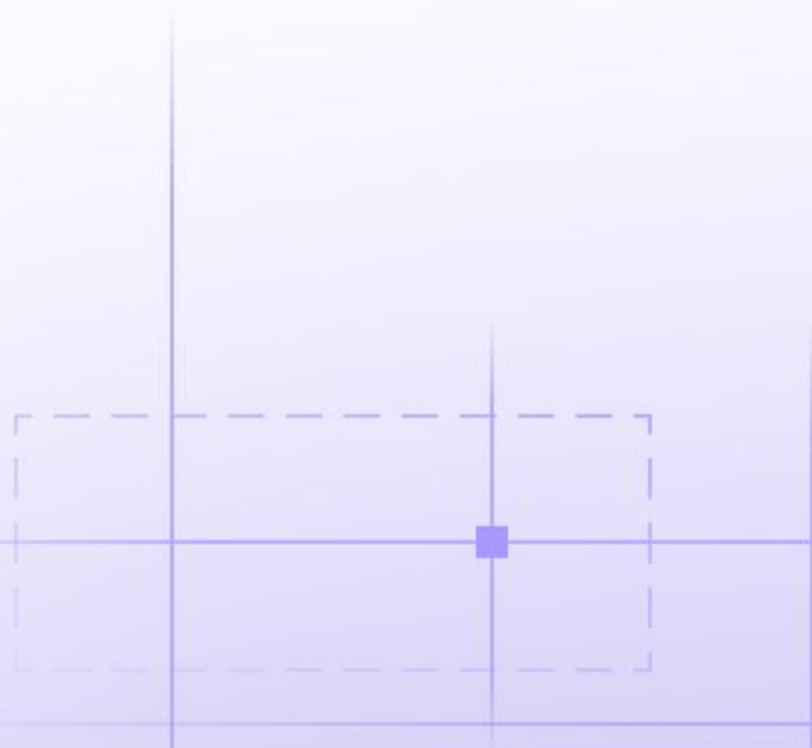
```
func sumThroughDefer(from int, numbers []int) (res int) {  
    defer func() {  
        sum := 0  
        for f := from; f < len(numbers); f++ {  
            sum += numbers[f]  
        }  
        res = sum  
    }()  
  
    return 0  
}
```

```
type deferClosure struct {  
    deferFn uintptr  
    from    int  
    numbers []int  
    res     *int  
}
```

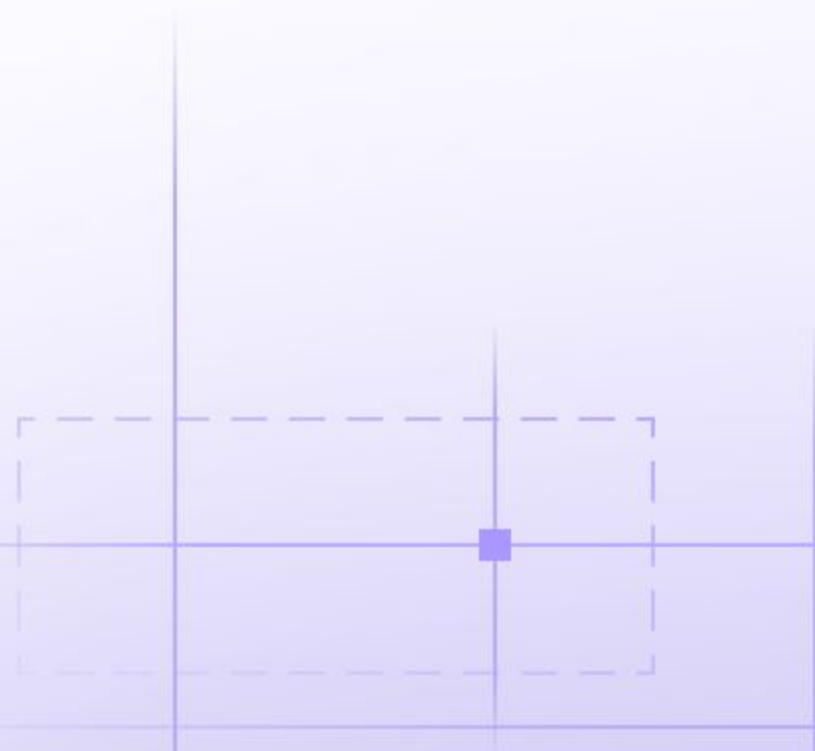
```
func sumThroughDefer(from int, numbers []int) (res int) {  
    defer func() {  
        sum := 0  
        for f := from; f < len(numbers); f++ {  
            sum += numbers[f]  
        }  
        res = sum  
    }()  
  
    return 0  
}
```

```
type deferClosure struct {  
    deferFn uintptr  
    from    int  
    numbers []int  
    res     *int  
}
```

Бенчмарк	Кол-во итераций	Время	Дельта
SumThroughDefer	1	10.07 ns	-26 % (-2.85 ns)
SumThroughDeferOpt	1	7.22 ns	



Бенчмарк	Кол-во итераций	Время	Дельта
SumThroughDefer	1	10.07 ns	-26 % (-2.85 ns)
SumThroughDeferOpt	1	7.22 ns	
SumThroughDefer	10	28.78 ns	-55 % (-16.01 ns)
SumThroughDeferOpt	10	12.77 ns	



Бенчмарк	Кол-во итераций	Время	Дельта
SumThroughDefer	1	10.07 ns	-26 % (-2.85 ns)
SumThroughDeferOpt	1	7.22 ns	
SumThroughDefer	10	28.78 ns	-55 % (-16.01 ns)
SumThroughDeferOpt	10	12.77 ns	
SumThroughDefer	100	229.10 ns	-67 % (-153.88 ns)
SumThroughDeferOpt	100	75.22 ns	



Defer

```
func sumThroughLambda(from int, numbers []int) (res int) {  
    l := func() {  
  
        for ; from < len(numbers); from++ {  
            res += numbers[from]  
        }  
    }  
  
    l()  
    return res  
}
```

Defer

```
func sumThroughLambda(from int, numbers []int) (res int) {  
    l := func() {  
        defer func() {}()  
  
        for ; from < len(numbers); from++ {  
            res += numbers[from]  
        }  
    }  
  
    l()  
    return res  
}
```

Бенчмарк	Кол-во итераций	Время	Дельта
SumThroughDefer	1	10.07 ns	+1% (+0.17 ns)
SumThroughLambda	1	10.24 ns	



Бенчмарк	Кол-во итераций	Время	Дельта
SumThroughDefer	1	10.07 ns	+1% (+0.17 ns)
SumThroughLambda	1	10.24 ns	
SumThroughDefer	10	28.78 ns	0% (+0.2 ns)
SumThroughLambda	10	28.92 ns	



Бенчмарк	Кол-во итераций	Время	Дельта
SumThroughDefer	1	10.07 ns	+1% (+0.17 ns)
SumThroughLambda	1	10.24 ns	
SumThroughDefer	10	28.78 ns	0% (+0.2 ns)
SumThroughLambda	10	28.92 ns	
SumThroughDefer	100	229.10 ns	0% (+0.23 ns)
SumThroughLambda	100	229.33 ns	



Причины просадки производительности

- ✓ Функция с `defer` не инлайнится



Defer

```
func (c *Cache) Get() float64 {  
    c.mtx.Lock()  
    defer c.mtx.Unlock()  
  
    return c.value  
}
```

Причины просадки производительности

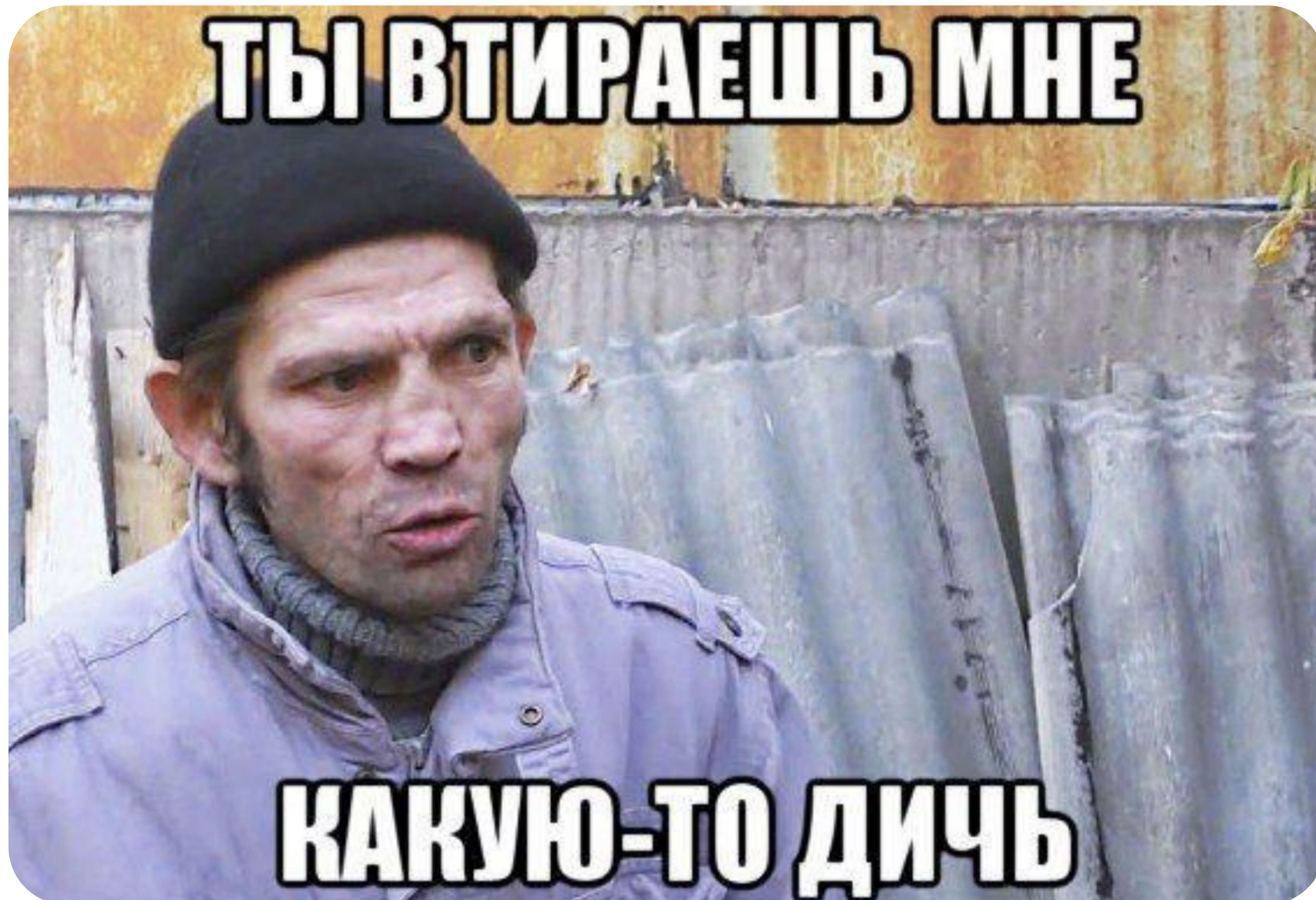
- ✓ Функция с `defer` не инлайнится
- ✓ `defer`-функция не инлайнится
- ✓ Генерация служебных данных (`deferBits`, `deferContextes`)
- ✓ Двойные прыжки по памяти при использовании модифицируемых переменных внутри контекста замыкания



Советы по оптимизации

- ✓ Не использовать defer
- ✓ Не модифицировать переменные из контекста замыкания





Общие советы по оптимизации

- ✓ Использовать PGO / ручной инлайнинг
- ✓ Меньше обобщённого кода, больше подсказок компилятору
- ✓ Разумно использовать defer/lambda на hot path
- ✓ Любые изменения кода контролировать прогоном бенчмарков

Важно соблюдать баланс между качеством кода и его производительностью



Репозиторий с бенчмарками



GitHub



Deckhouse Prom++

Prom++ – **оптимизированный**
Prometheus с потреблением
памяти до 10 раз меньше



Спасибо за внимание

 @deckhouse_news

 github.com/deckhouse

